

基于开源鸿蒙与昇腾边缘智能的 自主巡检机器人

设计文档

所在赛道与选项：A

摘 要

本项目面向工业巡检场景，设计并实现了一套全栈国产化的自主巡检机器人系统，用于代替人工完成区域遍历与工业仪表读数分析。系统以开源鸿蒙（OpenHarmony 5.0）为操作系统、以华为昇腾 310B NPU 为边缘算力，不使用 ROS 等境外机器人中间件，从硬件搭建、激光雷达驱动到 SLAM 建图、路径规划、仪表识别与大模型分析均自主实现，数据不出本体、断网可用。

系统采用三节点异构架构：每辆车由一块紫派和一块香橙派组成，紫派（OpenHarmony 5.0）负责激光感知、SLAM 导航与运动控制，香橙派（昇腾 310B）负责仪表检测、关键点定位与读数几何换算，并离线运行 DeepSeek 大模型生成巡检分析报告；香橙派识别到仪表后联动紫派减速，读数完成后恢复行进。鸿蒙平板（ArkTS）负责人机交互、实时可视化与多车总控。多机协同时，各车与平板通过华为分布式软总线共享同一份任务状态，按划分的子区域协同完成大区域全覆盖巡检。

实测表明，系统可在室内场景自主建图、导航避障与全覆盖遍历；仪表识别端到端延迟约 34 ms（约 29 FPS），支持一个画面内多块仪表同时读数与越限告警；端侧大模型可全程离线生成趋势分析报告，验证了国产软硬件承载完整机器人系统的可行性。

关键词：开源鸿蒙（OpenHarmony）；分布式软总线；昇腾边缘计算；同步定位与建图（SLAM）；全覆盖路径规划；工业仪表识别

目录

摘要	I
第一章 项目背景与目标	1
1.1 项目定位	1
1.2 工业巡检的需求与痛点	1
1.3 政策与技术背景	2
1.4 目标问题定义	2
1.5 意义与价值	3
第二章 系统总体设计	4
2.1 设计原则	4
2.1.1 国产自主与零外部依赖	4
2.1.2 视觉与导航解耦	4
2.2 系统总体架构	4
2.3 三大异构节点的职责	6
2.4 四层通信方式	6
2.5 端到端任务流程	9
2.6 关键技术选型	9
第三章 硬件平台与系统部署	12
3.1 整车硬件组成	12
3.2 运动主控 Purple Pi OH	12
3.3 视觉单元香橙派 AI Pro	12
3.4 RPLIDAR A2M8 激光雷达	13
3.5 电机、驱动板与供电	13
3.6 OpenHarmony 5.0 裁剪适配与烧录	14
3.7 板端工具与依赖库构建	15
3.8 三端部署与启动流程	15
3.9 真机调试经验	16

第四章 视觉感知与智能分析	17
4.1 技术路线	17
4.1.1 两段式读数路线的选择	17
4.1.2 处理流水线	17
4.2 自建数据集与模型训练	17
4.3 YOLOv5s 仪表检测	18
4.4 Simple Baselines 关键点检测	19
4.5 仪表读数的几何换算	20
4.6 多表检测与误检过滤	20
4.7 昇腾推理流水线	21
4.8 DeepSeek 大模型端侧部署	22
4.9 视频流与数据服务接口	23
4.10 数据采集、存储与分析	24
第五章 自主导航与运动控制	25
5.1 子系统结构	25
5.2 激光雷达驱动与统一感知链路	26
5.3 UDP-LCM 桥接与 9 字节控制协议	26
5.4 SLAM 建图与地图管理	27
5.5 定位与位姿更新	27
5.6 A* 全局规划与 DWA 局部避障	28
5.7 全覆盖路径规划与多机分区执行	28
5.8 车间协同避障 COOP_AVOID	28
5.9 差速轮控制与串口安全	29
5.10 运行状态管理与接口边界	29
第六章 多机协同	30
6.1 协同场景与设计选择	30
6.2 分布式软总线的三层能力	30
6.3 FleetMission 共享任务黑板	31
6.4 设备互信与 PIN 认证	32
6.5 车载无界面 agent	34

6.6	双车分区覆盖与协同避障	34
第七章	平板控制应用	38
7.1	应用总体架构	38
7.2	统一传输层与多车保活	38
7.3	地图渲染与坐标换算	39
7.4	视频流与读数展示	40
7.5	任务编排与交互设计	40
第八章	性能优化	41
8.1	数据通路与瓶颈定位	41
8.2	地图位打包压缩 (ZMAP1)	41
8.3	推理流水线提速	42
8.4	端到端性能指标	42
第九章	系统运行实测	43
9.1	自主建图与导航实测	43
9.2	全覆盖遍历实测	44
9.3	仪表识别精度与实时性	45
9.4	DeepSeek 端侧分析实测	46
9.5	多机协同实测	46
第十章	创新点	48
10.1	创新点总览	48
10.2	国产化与自主可控	49
10.3	异构架构与通信协议设计	50
10.3.1	双单元物理解耦架构	50
10.3.2	四层通信方式与 9 字节控制协议	50
10.3.3	ZMAP1 位打包压缩地图	51
10.4	自主导航算法链	51
10.4.1	多分辨率扫描匹配 SLAM	51
10.4.2	KLD 自适应粒子滤波定位	51
10.4.3	A* 代价规划与 DWA 局部避障	51

10.4.4 多策略全覆盖路径规划	51
10.5 多机协同创新	51
10.5.1 DDO 共享任务黑板	51
10.5.2 幂等的车载协同 agent	52
10.5.3 空间分区与 COOP_AVOID 协同避障	52
10.6 视觉与端侧大模型的结合	52
10.6.1 两段式可解释读数路线	52
10.6.2 视觉与大模型分时共享 NPU	52
10.6.3 端侧大模型工程化	52
第十一章 相关工作与对比	53
11.1 现有方案概况	53
11.2 技术维度对比	53
11.3 差异化特征	53
11.4 政策契合度	54
第十二章 总结与展望	55
12.1 工作总结	55
12.2 局限性与改进方向	55
12.3 后续研究方向	56
参考文献	57
附录 A 术语表	58

第一章 项目背景与目标

本章说明项目的应用背景、立项依据与要解决的问题，并给出系统的目标定义。

1.1 项目定位

本项目设计并实现一台面向工业巡检场景的自主移动机器人，目标是：代替人工在工厂、变电站、机房等场景中自主遍历全区域，自动读取并分析工业仪表读数，并将过程与结果实时呈现在管理终端上。为此，系统需同时具备三类能力：

1. 自主移动与全覆盖。在无人工遥控、无预先地图的条件下自主完成建图与定位（SLAM），规划一条不漏扫、低重复的巡检路径，并在行进中实时躲避障碍。
2. 视觉读数与分析。在移动中实时识别画面中的工业仪表，定位表盘关键点并换算出指针读数，判断是否越限告警，并能调用大模型对历史数据生成趋势分析报告。
3. 多机协同与统一展示。多台机器人在同一坐标系下分区协作、协同覆盖大区域；所有机器人的位姿、进度与识别结果实时汇聚到同一台平板终端。

上述能力均由本项目自主实现，并运行在国产软硬件生态之上。

1.2 工业巡检的需求与痛点

在电力、石化、冶金、轨道交通、数据中心等场景中，抄录现场大量指针式仪表（压力表、油温表、SF₆ 气体密度表等）是巡检作业中占用人力最多、也最易出错的环节。此类仪表无源可靠、抗干扰、防爆性好，短期内难以被数字传感器全面替代。传统人工巡检存在三方面问题：

- 效率低、成本高。据行业实践数据，设备密集区域人工完成全区域巡检约需 300 min，智能系统仅需约 120 min，效率相差 2.5 倍以上。
- 安全风险大。巡检常涉及高压、高温、有毒、易燃易爆等危险环境，极端天气与夜间作业时风险更为突出。
- 数据难以追溯。纸质记录形成信息孤岛，历史数据无法用于趋势研判与预测性维护。

因此，以机器人替代人工巡检已成为行业趋势。市场研究也印证了这一方向：多家机构预测全球巡检机器人市场 2025 年已达数十亿美元规模，并将以约 13%–17% 的

年复合增长率增长至百亿美元以上（表 1.1），其主要驱动力正是石化、电力等高危场景的自动化巡检需求。

表 1.1 全球巡检机器人市场规模预测

研究机构	2025 年规模	远期规模	CAGR
Market Research Future	47.53 亿美元	225.4 亿美元（2035）	16.84%
Future Market Insights	32 亿美元	117 亿美元（2035）	13.9%
Maximize Market Research	67.2 亿美元	161.6 亿美元（2032）	13.2%

1.3 政策与技术背景

本项目的技术路线与国家产业政策和近年技术成熟度直接相关。

政策层面，与本项目最相关的有三点：（1）2021 年工信部等 15 部门印发的《“十四五”机器人产业发展规划》将安防巡检、危险环境作业类机器人列为重点方向；（2）2025 年国务院《政府工作报告》首次写入“具身智能”与“智能机器人”，北京、深圳、杭州等地随后出台三年行动方案；（3）信创替代持续推进，相关政策提出 2027 年央企国企关键软硬件全面国产替代的目标，覆盖操作系统、中间件等领域。本项目的操作系统（开源鸿蒙）、AI 算力（华为昇腾）与开发语言（ArkTS）均为国产自主，正对应上述方向。

技术层面，三项条件近年趋于成熟，使本系统可行：其一，激光 SLAM、粒子滤波定位、A* 规划等导航算法已可在嵌入式平台实时运行，本项目的难点与价值在于不借助 ROS 生态、在 OpenHarmony 上自主实现完整算法栈；其二，以昇腾 310B 为代表的边缘 NPU 使 YOLO 检测、关键点回归等视觉模型可在机器人本体实时推理，无需云端；其三，DeepSeek-R1-Distill 等蒸馏小模型的出现，使大模型可在边缘 NPU 上离线运行，机器人本体可直接生成自然语言分析报告。这三项条件使“自主移动 + 视觉读数 + 智能分析”的全部能力可以完全由国产端侧软硬件承载。

1.4 目标问题定义

综合上述背景，本项目要解决四个相互关联的问题：

1. 自主导航：在未知、无 GPS 的室内环境中，依靠激光雷达与轮控里程估计自主完成建图与定位，并在已建地图上实现全局定位与避障导航，且不依赖 ROS 及任何境外机器人中间件。
2. 全覆盖巡检：在含非凸形状与内部障碍的栅格地图上，规划一条完备、低重复的全覆盖路径；并能把一片大区域均衡划分给多台机器人协同覆盖。

3. 仪表读数：在边缘 NPU 上实时检测工业仪表、定位表盘关键点，通过几何换算得到指针读数，支持一个画面内多块仪表同时识别与越限告警。
4. 多机协同：不依赖中心服务器，使多台设备在同一可信网络中共享一致的任务状态，并将全局态势实时呈现给管理者。

后续各章的设计与实现均围绕这四个问题展开。

1.5 意义与价值

在技术层面，本项目验证了国产操作系统承载复杂机器人系统的可行性：在 Open-Harmony 5.0 上不借助 ROS，自主实现雷达驱动、SLAM、规划与控制的完整闭环；同时验证了在鸿蒙分布式软总线上用共享任务状态组织多机协同、以及在同一块昇腾 NPU 上让实时视觉推理与端侧大模型分时共存的工程方案。

在应用层面，全栈国产自主的路线使系统可直接服务于电力、能源等对自主可控有刚性要求的场景；自主全覆盖巡检与实时读数可替代高危、低效的人工抄表作业；模块化的架构也便于扩展红外测温、气体检测等更多巡检模态，具有清晰的产业化前景。

第二章 系统总体设计

本章介绍系统的设计原则、总体架构、节点分工与通信方式，为后续各子系统章节提供整体背景。

2.1 设计原则

系统实现之前确立了两条设计原则，它们决定了代码的物理边界与各部分的协作方式。

2.1.1 国产自主与零外部依赖

第一条原则是全栈国产化与算法自研，具体落实为三个“不依赖”：

- 不依赖 ROS 及任何境外机器人中间件。SLAM 建图、粒子滤波定位、A* 规划、全覆盖算法、差速轮控等功能全部自主实现，仅以轻量级 LCM 作为紫派内部进程通信总线。
- 不依赖境外操作系统与云端。操作系统采用开源鸿蒙 OpenHarmony 5.0，应用层采用国产 ArkTS 语言，AI 算力采用华为昇腾 NPU，大模型采用端侧离线的 DeepSeek，全流程数据不出本体、断网可用。
- 不依赖外购整机。硬件结构、电机与雷达接线、供电系统均自主搭建调试。

2.1.2 视觉与导航解耦

第二条原则是将视觉感知与运动导航在物理上完全解耦，分别承载于两块独立的计算单元——香橙派负责视觉，紫派负责导航。二者不共享进程、不共享算力，仅通过明确的网络接口交互。这里的“解耦”约束的是部署与算力边界，并不排斥经受控接口交换状态或速度约束。该设计带来三方面收益：算力专用化（NPU 专注视觉推理，CPU 专注实时 SLAM 与轮控，互不抢占）；故障隔离（一侧故障不影响另一侧）；独立演进（视觉模型与导航算法可各自迭代）。

此外，由于三端代码分别用 C/C++、Python、ArkTS 编写且互不编译依赖，所有跨设备接口（字节布局、端口、坐标单位等）由一份共享的接口文档统一约定，任何协议修改须三端同步，以此保证异构系统的集成一致性。

2.2 系统总体架构

在上述原则下，系统采用“鸿蒙平板 + 多辆双计算单元小车 + 两类协同通道”的总体架构，如图 2.1 所示。每辆车由一块紫派和一块香橙派组成，平板统一下发任务并汇总各车状态。

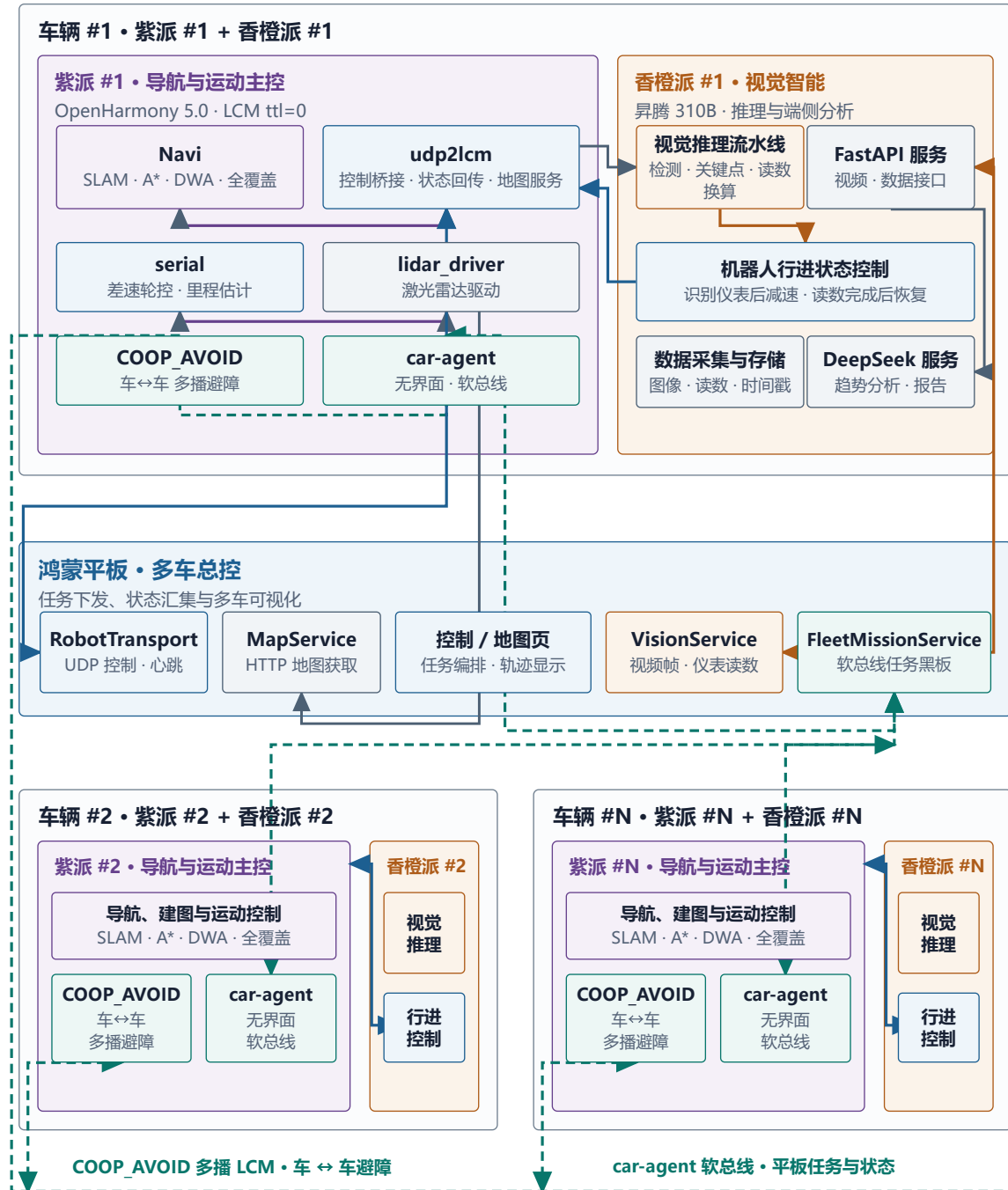


图 2.1 系统总体架构图

图 2.1展示了系统的五层关系：（1）平板与车辆——鸿蒙平板作为多车总控终端，每辆车由紫派运动主控和香橙派视觉单元组成；（2）平板到各车的控制与数据链路——平板经 UDP 5001 端口下发 9 字节控制命令，紫派每 500 ms 回传携带位姿的心跳，地图文件经 HTTP 8000 端口获取，香橙派经 WebSocket 推送视频帧与识别结果；（3）紫派内部运动闭环——lidar_driver（雷达）、Navi（SLAM 与规划）、serial（轮控）、udp2lcm（协议桥接）通过 LCM 总线协作，构成“感知 → 决策 → 执行”闭环；（4）车内视觉-运动联动——香橙派识别到仪表后向同车紫派下发减速指令，读数完成或目标离开后下发恢复指令，紫派执行运动控制并回传当前行进状态；（5）多机协同——每辆车的 car-agent 以无界面服务接入平板的分布式软总线任务黑板，各车的 COOP_AVOID 通过车间多播 LCM（ttl=1）直接交换避障状态。图中完整展开车辆 #1，并以相同接口展示车辆 #2 与车辆 #N。

2.3 三大异构节点的职责

表 2.1汇总了三大节点的设备、技术栈与核心职责。分工取决于各自的硬件特性：紫派 RK3566 的 CPU 适合承载实时性要求较高的 SLAM 与控制任务；香橙派昇腾 310B 的 NPU 面向 AI 推理设计；鸿蒙平板适合触控交互与可视化。

2.4 四层通信方式

系统的通信由四层在不同范围、不同协议上工作的通道构成，如图 2.2所示。

第 (1) 层 设备内总线 LCM——紫派内部进程通信。LCM 是基于 UDP 组播的轻量级发布/订阅中间件；主总线 ttl=0（仅本机可见），保证两台车的内部信道相互隔离。

第 (2) 层 控制通道 UDP 5001——平板与车的实时控制链路。采用 9 字节定长二进制协议；选 UDP 而非 TCP 的原因是控制命令要求低延迟且可容忍偶发丢包（下一帧及时补充）。

第 (3) 层 数据通道 HTTP/WebSocket——数据传输与车内状态交互。紫派经 HTTP 8000 端口提供地图文件；香橙派经 WebSocket 推送视频帧与 JSON 读数；同车的香橙派和紫派经车内网络交换减速、恢复指令与行进状态。

第 (4) 层 协同通道——多机状态同步。包含两条互补链路：华为分布式软总线承载共享任务状态（协调态），车与车的多播 LCM（COOP_AVOID，ttl=1）用于执行层的直接协商让行。

表 2.1 三大异构节点职责与技术栈

节点	物理设备		操作系统	主要语言		核心职责
鸿蒙平板	HarmonyOS 平板	平	HarmonyOS	ArkTS ArkUI	/	人机交互、建图触发、目标点/区域下发、地图与识别结果可视化、多机总控
紫派主控	Purple (RK3566)	Pi	OpenHarmony 5.0	C/C++ Python ArkTS	+ +	雷达感知、SLAM、导航、全覆盖、差速轮控、HTTP 地图服务、多机协同、接收减速或恢复指令并回传行进状态
香橙派视觉	Orange Pi AI Pro (昇腾 310B)		Linux (CANN)	Python + C++ + MindSpore		摄像头实时推理、读数几何换算、越限告警、数据存储、DeepSeek 端侧分析、识别仪表后控制机器人行进状态

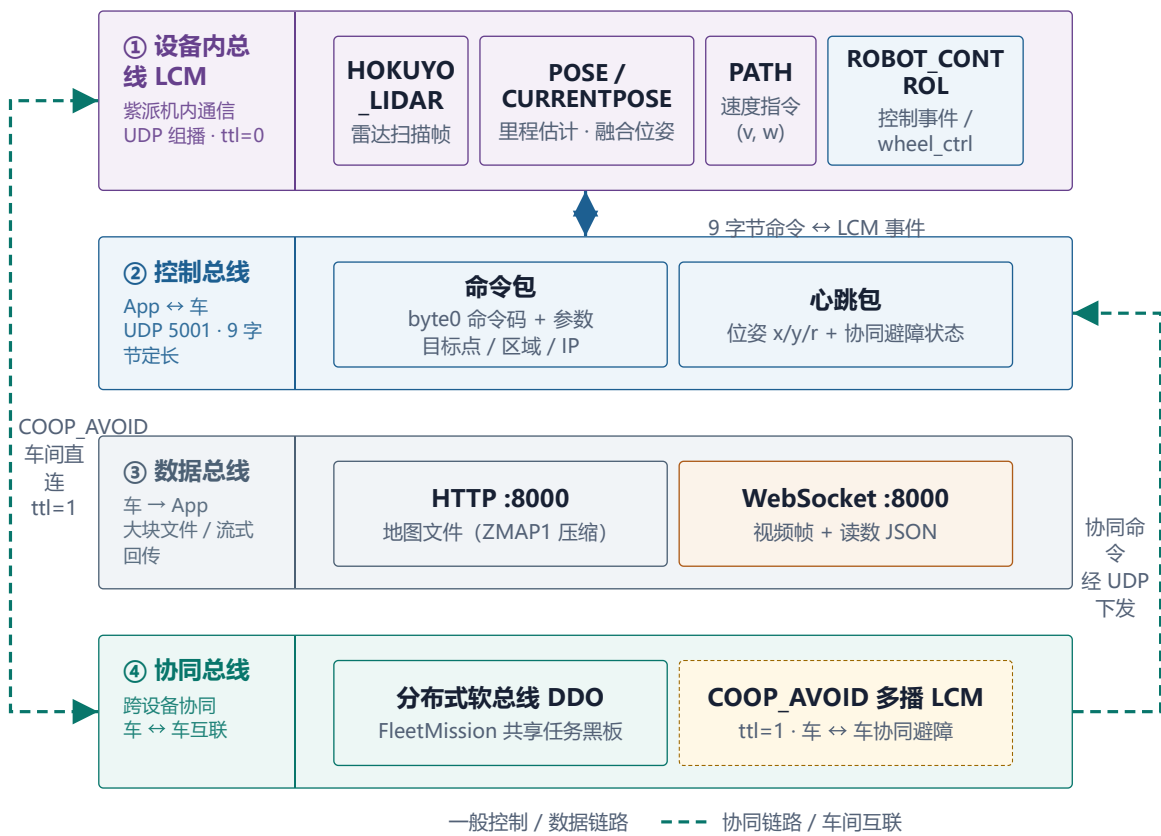


图 2.2 四层通信通道数据流图

2.5 端到端任务流程

图 2.3 给出一次完整单车巡检任务的时序，展示了数据如何在各节点间流动。

一次典型巡检任务分为五个阶段：

- (1) 建图启动：操作者在平板点击“开始建图”，平板发出 UDP 命令 'm'，紫派启动 SLAM，以激光扫描配合轮控里程估计逐帧拼接地图，并以 500 ms 周期回传实时位姿。
- (2) 遥控探索：建图阶段人工遥控车辆遍历环境，平板持续发送运动命令并保活（满足“至少每秒一帧”的安全约定，否则紫派在 3 s 后急停）。
- (3) 结束建图与取图：紫派保存地图并生成 ZMAP1 压缩图，平板经 HTTP 拉取后在本地解压、换算坐标并渲染。
- (4) 覆盖巡检：操作者在地图上框选区域或点选目标点，紫派以 BCD 牛耕分解规划全覆盖路径，A* 全局规划、DWA 局部避障，走完全程并持续回传位姿与进度。
- (5) 视觉与分析（全程并行）：平板经 WebSocket 接收香橙派推送的叠加检测框与读数的视频流；香橙派识别到仪表后向同车紫派下发减速指令，读数完成或目标离开后下发恢复指令，紫派执行后回传行进状态。操作者还可请求生成分析报告，香橙派暂停视觉推理以释放 NPU，调用端侧 DeepSeek 对历史读数生成趋势分析报告。

该流程体现了系统的一个关键特征：运动链路（平板与紫派）与视觉链路（平板与香橙派）在时间上并行、在物理部署上解耦；同车的香橙派与紫派通过明确的网络接口交换减速、恢复指令和行进状态，使仪表识别与运动控制形成闭环。

2.6 关键技术选型

- 内部通信选 LCM 而非 ROS：LCM 轻量、无中心节点，可在 OpenHarmony 上零依赖移植；ROS 生态庞大、移植成本高，与国产自主原则冲突。
- 控制协议选 9 字节定长 UDP：紧凑、解析无歧义、跨语言实现简单，适合高频小包控制。
- 地图传输选 HTTP + 位打包压缩：HTTP 通用易实现，配合 ZMAP1 位打包可将地图体积压至约 1/8。
- 视频流选 WebSocket + JPEG：实现简单，可与读数 JSON 共用同一连接，延迟低，无需额外播放器组件。

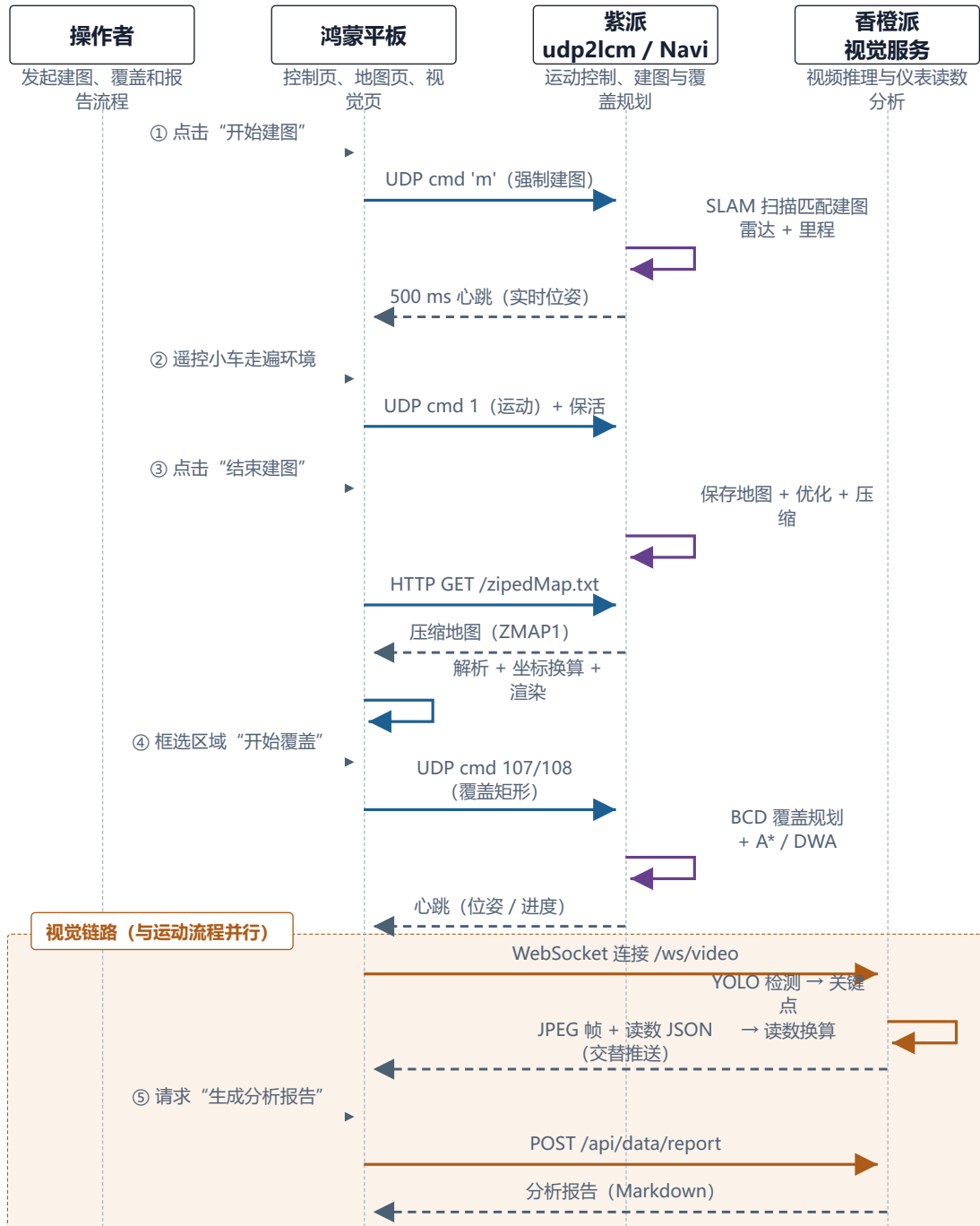


图 2.3 单车巡检任务端到端时序图

- 多机协同选软总线共享状态：以跨设备共享任务状态替代远程拉起页面传参，消除了把 Ability 生命周期当作远程调用使用的脆弱性。

上述选型共同逻辑是：在满足功能与实时性要求的前提下，优先选择轻量、自主、易于在国产生态上落地的技术路线。

第三章 硬件平台与系统部署

本章介绍整车硬件组成、各计算单元与传感器选型，以及 OpenHarmony 5.0 的裁剪适配、依赖库构建与三端部署流程。

3.1 整车硬件组成

整车由“双计算单元 + 感知 + 执行 + 供电”四部分组成，对应第 2.1.2 节视觉-导航解耦的物理实现，如表 3.1 所示。紫派通过两条 USB 串口分别连接雷达（ttyUSB0）和底盘（ttyUSB1），香橙派连接 USB 摄像头，两块板卡经车内网络交换减速、恢复指令与行进状态，并与平板接入同一局域网。

表 3.1 整车硬件组成

部件	选型	接口/连接	角色
运动主控	紫派 Purple Pi (RK3566)	—	运行 OpenHarmony 5.0，承载导航四进程并执行车速控制
视觉单元	香橙派 Orange Pi AI Pro (昇腾 310B)	网口 (192.168.1.5)	视觉推理、行进状态控制与 DeepSeek 大模型
激光雷达	SLAMTEC RPL-IDAR A2M8	/dev/ttyUSB0 (115200)	360° 二维测距，SLAM 感知输入
底盘	两驱动轮 + 万向轮 差速底盘 + 电机驱动板	/dev/ttyUSB1 (115200, 8N1)	执行运动；轮控进程按速度指令积分生成里程估计
摄像头	USB 摄像头	/dev/video0	仪表画面采集
交互终端	鸿蒙平板	Wi-Fi	总控与可视化

3.2 运动主控 Purple Pi OH

运动主控选用 Purple Pi OH 开发板，核心处理器为 RK3566（四核 Cortex-A55），兼容 OpenHarmony 5.0，可满足导航进程、地图服务与串口通信的并行运行需求。板卡尺寸约 85 mm × 56 mm，提供千兆网口、USB 2.0/3.0 与 40 pin 扩展排针，可同时连接雷达、底盘串口与局域网，降低了整车布线复杂度（图 3.1）。主控单独配置 5 V 供电模块，避免电机启动、雷达旋转造成主控端压降。

3.3 视觉单元香橙派 AI Pro

视觉单元选用香橙派 Orange Pi AI Pro，搭载华为昇腾 310B 处理器，内置 NPU，可在板端实时运行 YOLO 检测、关键点检测模型以及端侧 DeepSeek 大模型。板卡运

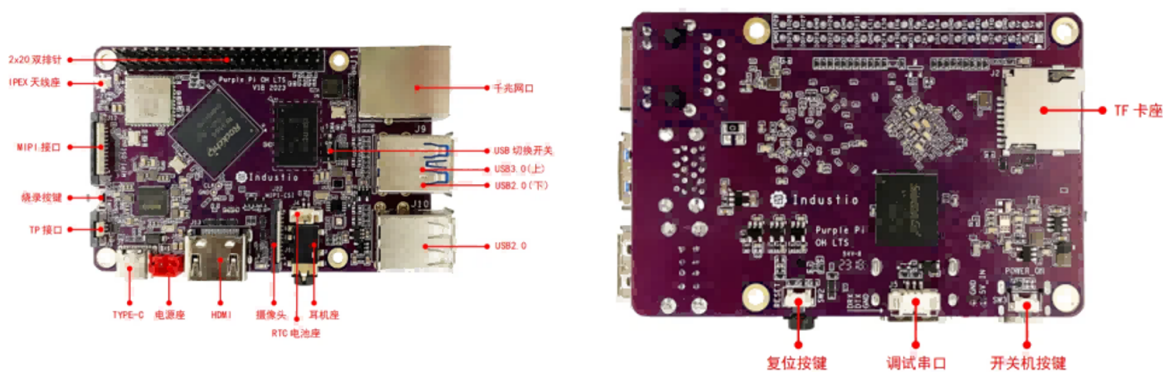


图 3.1 Purple Pi OH 正反面接口

行基于 CANN 的 Linux 系统，提供完整的 ACL 运行时与 ATC 模型编译工具链，经千兆网口（固定 IP 192.168.1.5）与紫派、平板互联（图 3.2）。

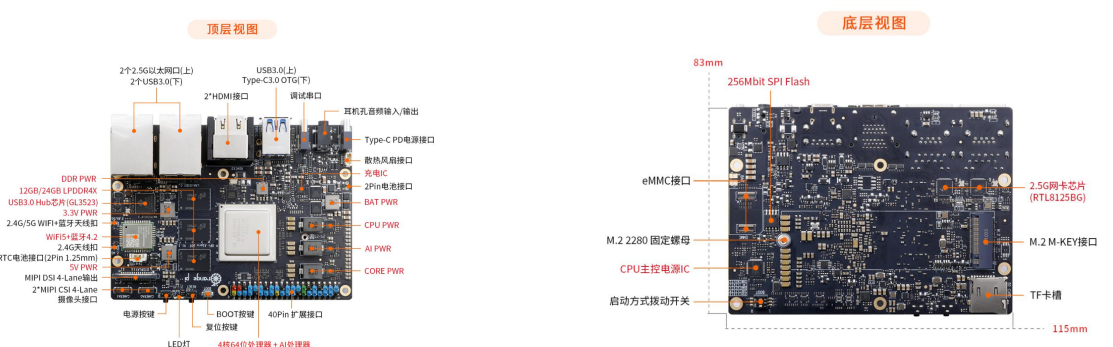


图 3.2 香橙派 AI Pro 正反面接口

3.4 RPLIDAR A2M8 激光雷达

激光感知采用 SLAMTEC RPLIDAR A2M8，输出 360° 二维测距数据，典型旋转频率 10 Hz，典型角分辨率约 0.45°，满足室内建图与避障需求（图 3.3）。该雷达以单个激光采样点为基本单位输出，驱动程序连续读取采样点并在角度回绕处切分为完整扫描帧；相比按整圈回调的方案，逐点输出便于驱动层做角度排序、异常点过滤与帧边界判断，与 lidar_driver 的实现方式相契合。

3.5 电机、驱动板与供电

底盘执行机构采用兆威 ZWMD032032-33-01 行星减速电机（12 V，减速比 32.8），可为低速巡检底盘提供较高输出力矩，关键参数如表 3.2 所示。电机驱动板未采用商用整机控制器，而是围绕 OpenHarmony 主控与底盘需求自行设计：支持双轮差速控制，通过串口接收轮控进程下发的方向与速度指令（图 3.4）。供电上，电机侧使用 12 V 电源模块、主控侧使用 5 V 电源模块，二者按功能域分离，降低了电机瞬态电流

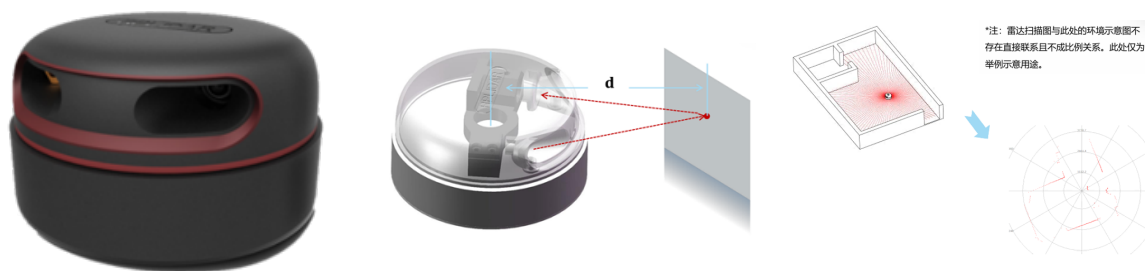


图 3.3 RPLIDAR A2M8 实物、工作原理与扫描效果

对主控与串口通信的影响，也便于排查压降、重启与外设掉线问题。

表 3.2 驱动电机关键参数

参数	数值
直径 / 长度	32 mm / 81.6 mm
减速比	32.8
额定电压	12 VDC
齿轮箱额定容许力矩	60000 gf.cm
空载数据	转速 164.40 rpm；最大电流 235 mA
负载数据	转速 147.87 rpm；最大电流 715 mA；转矩 2518.93 gf.cm

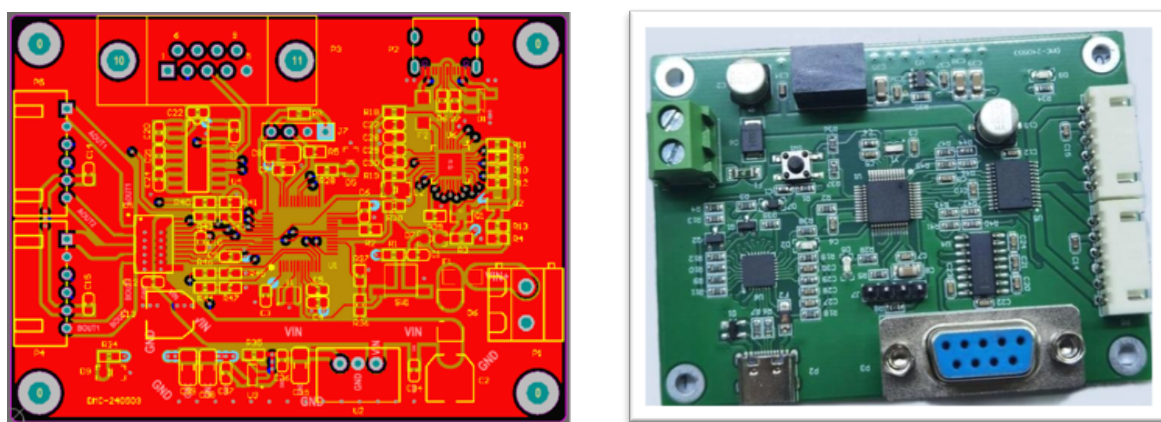


图 3.4 电机驱动板卡设计与实物

3.6 OpenHarmony 5.0 裁剪适配与烧录

紫派运行的是面向本项目裁剪适配的 OpenHarmony 5.0 镜像。构建流程为：在宿主机配置编译环境并安装 repo 工具；拉取 OpenHarmony 5.0 源码后打入 Purple Pi OH 板级补丁，使内核与设备配置匹配 RK3566；修正两处 64 位系统配置后执行编译，生成的镜像经 RKDevTools 烧录至板卡，烧录后验证图形界面与网络连接正常（图 3.5）。

适配过程中解决的两个关键问题值得说明：（1）64 位默认配置栈空间不足，表现为设置无法启动、WLAN 无法开启，需在板级配置中将 device_stack_size 设为

8388608; (2) 受 SELinux 沙箱限制, WLAN 操作存在权限错误, 需在系统沙箱配置中补充 WLAN 相关访问权限, 使网络初始化正常完成。

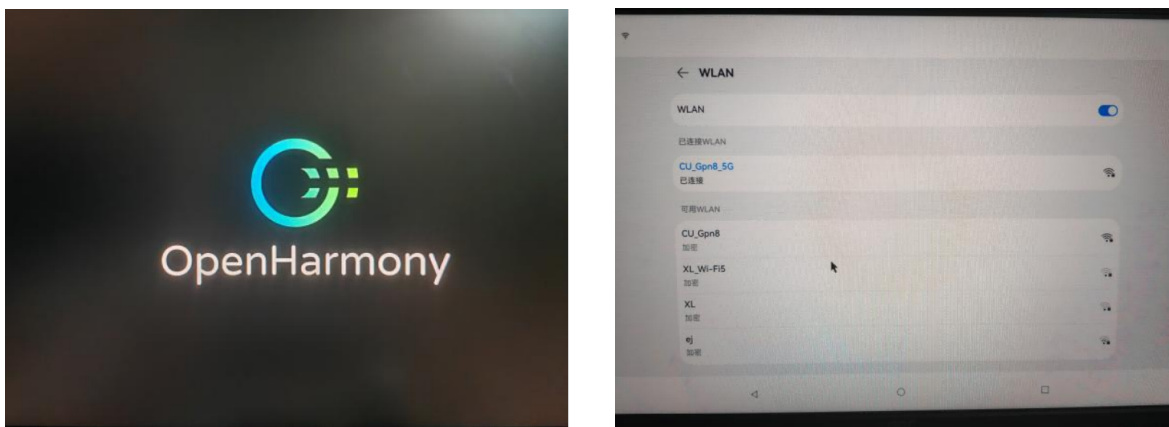


图 3.5 Purple Pi OH 烧录完成后的连接检查与图形界面

3.7 板端工具与依赖库构建

多机地图同步要求紫派能在板端直接发起 HTTP 请求 (从车需拉取主车地图), 而 OpenHarmony 默认的轻量化 toybox 环境不包含完整 wget。本项目基于交叉编译工具链, 依次构建 zlib、openssl 与 wget, 并解决了 GNU 工具链产物依赖 ld-linux-aarch64.so 而板端使用 ld-musl-aarch64.so.1 的动态链接器差异问题, 使 wget 可在板端原生运行。

紫派内部的雷达、导航、轮控与桥接进程均通过 LCM 通信, LCM 及其底层依赖 glib 同样需交叉编译至板端。编译时裁剪了 LCM 的 Python 绑定, 仅保留 C/C++ 核心库, 并通过自定义测试消息完成了宿主机收发验证, 随后将库文件部署至板端, 四个控制进程均动态链接成功。相比引入 ROS, LCM 体积小、消息模型简单, 更适合资源受限的 RK3566 板端。

3.8 三端部署与启动流程

三台设备均有脚本化的部署与启动流程。紫派的机器人栈统一部署在 /data/test 目录, 由 test.sh 一键拉起四个核心进程:

```
1 ./test.sh run      # 检查 /dev/ttyUSB* 数量为 2, 依次启动:
2                   #   lidar_driver / navigation / serial / udp2lcm
3 ./test.sh stop     # 向四进程 + python 发送 SIGINT 停止
4 ./test.sh clean    # 清理 /data/test 下日志与历史地图文件
```

香橙派侧启动昇腾推理服务:

```
1  ssh HwHiAiUser@192.168.1.5
2  cd ~/simple_baselines/web/backend
3  source /usr/local/Ascend/ascend-toolkit/set_env.sh
4  python3 main.py      # FastAPI 监听 0.0.0.0:8000
```

其中 `set_env.sh` 加载昇腾工具链（ACL 运行时、ATC 编译器等）的环境变量，是 OM 模型在 NPU 上加载推理的前提。

3.9 真机调试经验

真机调试中积累了两条关键经验。其一，边缘 AI 设备对供电质量敏感：香橙派曾出现系统负载虚高、ACL 加载模型即崩溃重启的现象，最终定位为 NPU 满载推理时供电不足，更换稳定电源并冷启动后故障消除。其二，OpenHarmony 默认 30 s 息屏会影响无人值守运行，启动脚本在运行前执行 `power-shell wakeup` 与超时设置作为保活措施，停止时恢复默认。

整车硬件、接线、供电与 OpenHarmony 5.0 镜像均由本项目自主搭建与适配，与自研软件共同构成完整的软硬件系统。

第四章 视觉感知与智能分析

本章介绍运行在香橙派昇腾 NPU 上的视觉子系统：将摄像头画面逐步转化为结构化的仪表读数，并利用端侧大模型对历史读数生成趋势分析报告。

4.1 技术路线

4.1.1 两段式读数路线的选择

指针式仪表读数需回答两个问题：仪表在画面中的位置，以及指针指向量程的百分比。一种朴素方法是用端到端网络直接回归读数数值，但在工业场景下存在三点不足：画面中可能出现多块仪表，端到端方法难以处理可变数量目标；仪表的位置、大小、角度变化大，直接回归对姿态的鲁棒性不足；可解释性差，读数错误时无法定位原因。

为此，本项目采用两段式路线，将问题分解为两个子任务：

1. 目标检测：用 YOLOv5s 在整幅画面中定位每块仪表的边界框，解决位置与数量问题。
2. 关键点定位与几何换算：将每块仪表的 ROI 裁剪后送入 Simple Baselines 关键点网络，定位表盘的四个语义关键点（圆心、指针尖、零刻度、满刻度），再由纯几何方法换算读数百分比。

该路线的核心原则是：深度学习只负责定位，读数交给确定性的几何公式完成。这使读数过程可解释、可标定、可复现，适配不同量程的仪表只需更换标定参数。

4.1.2 处理流水线

完整流水线为：USB 摄像头抓帧 → YOLOv5s 检测仪表框 → 裁剪 ROI → 关键点检测（热力图）→ 解码出 4 个关键点坐标 → 角度几何换算读数 → 渲染叠加 → JPEG 编码 → WebSocket 推送至平板；同时读数入库，供大模型按需分析。每个模型均经华为 ATC 工具转换为昇腾原生 .om 格式在 NPU 上推理；整条流水线运行在独立后台线程中，配合帧队列与跳帧策略保证实时性。

4.2 自建数据集与模型训练

视觉模型的训练数据由项目成员围绕目标仪表（以压力表为主）自行采集与标注，覆盖不同光照、拍摄角度与表盘背景。标注包含两类标签：检测标签（每块仪表的边界框，单类别）与关键点标签（圆心、指针尖、零刻度、满刻度 4 个关键点坐标）。

模型训练基于成熟公开基线改造而非从零搭建：检测沿用 YOLOv5s 结构并改造为单类别输出；关键点检测沿用 Simple Baselines，输出通道改造为本项目定义的 4 个语义关键点。训练完成后经 ATC 转换为 .om 格式，并配置 AIPP 把色彩空间转换、归一化等预处理交由 NPU 硬件完成。此外，利用留存的样图离线运行推理并断言读数误差，作为模型迭代时的精度回归测试。

4.3 YOLOv5s 仪表检测

YOLO 系列单阶段检测器将检测转化为一次性回归问题：一次前向传播即在整幅图的网格上预测所有目标框的位置、大小与置信度，推理速度快，适合机器人实时场景。本项目选用其中的轻量版本 YOLOv5s，在精度与速度间取得适合边缘设备的平衡，检测流程如图 4.1 所示。

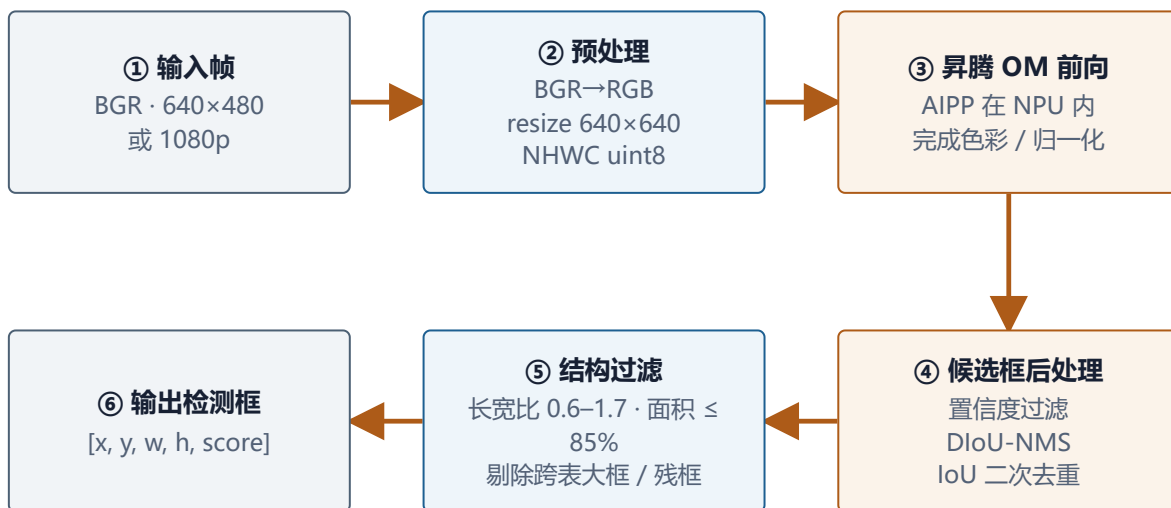


图 4.1 YOLOv5s 仪表检测流程图

检测流程的关键环节如下：

- 预处理：摄像头帧统一转换为 RGB 并缩放至 640×640（采用最近邻插值，检测任务对像素级精度不敏感，速度更快）。色彩转换与归一化通过 AIPP 交由 NPU 硬件完成，CPU 仅提供 uint8 数据。
- 置信度过滤：多表场景下阈值降至 0.10 以提高召回率（避免漏检），误检由后续多级过滤剔除。
- DIoU-NMS 去重：相比传统 IoU，DIoU 额外考虑两框中心点距离，对密集目标去重效果更好；其后再加一道更严格的二次 IoU 去重，消除同一仪表的多个重叠框。

- 长宽比与面积过滤：真实仪表长宽比通常在 0.6–1.7 之间、面积不超过全图 85%，据此剔除跨表大框与残框。

上述层层递进的过滤体现了“模型高召回、工程后处理高精度”的思路：模型尽可能多检出候选，工程后处理逐层去伪存真，最终输出干净的仪表框列表 $[x, y, w, h, \text{score}]$ ，进入关键点检测阶段。

4.4 Simple Baselines 关键点检测

第二阶段是在每块仪表的 ROI 内定位 4 个关键点的精确像素坐标。网络不直接回归坐标值，而是为每个关键点输出一张二维热力图——每个像素的值表示关键点出现在该位置的概率，取响应最大的像素作为关键点位置。热力图表示把“精确定位一个点”转化为“在概率图上找峰值”，更易学习，对位置噪声也更鲁棒。检测与解码流程如图 4.2 所示。

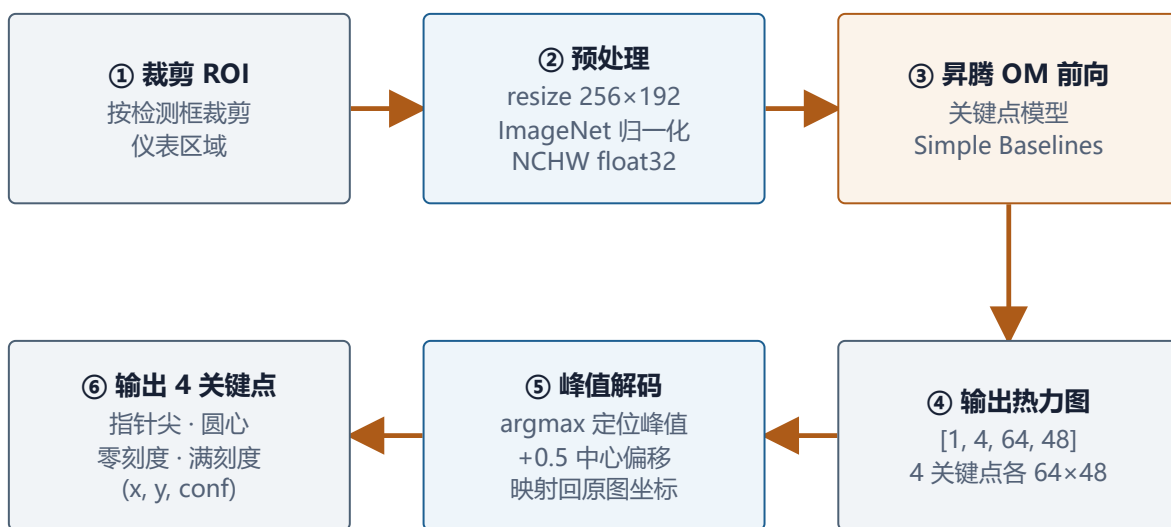


图 4.2 关键点检测与热力图解码流程图

关键环节如下：

- 预处理：输入为 256×192 的 NCHW float32 张量，按 ImageNet 均值与标准差归一化，保持骨干网络预训练特征分布一致。
- 热力图输出：网络输出 4 张 64×48 热力图，分别对应 4 个关键点；分辨率低于输入以降低计算量，解码时再映射回高分辨率。
- 峰值定位与坐标映射：对每张热力图用 argmax 取响应最大的格位及置信度，按比例放大回 ROI 像素坐标（峰值格加 0.5 中心偏移以提升亚像素精度），再叠加 ROI 在全图中的偏移，最终得到 4 个 (x, y, conf) 三元组。

4.5 仪表读数的几何换算

从 4 个关键点到读数的换算是视觉子系统可解释性的核心。将表盘抽象为圆，圆心为 C ：从圆心出发有三条射线——指向零刻度 Z （对应 0%）、指向满刻度 F （对应 100%）、指向指针尖 P （当前读数）。读数的本质是：指针射线 CP 落在零刻度射线 CZ 到满刻度射线 CF 所构成扇形夹角中的百分比。

实现时需处理两个方向问题。坐标方向上，图像坐标系 y 轴向下，计算各点相对圆心的向量时对 y 分量取负以转回数学坐标系，再用 atan2 求极角（ atan2 能正确处理四个象限并避免除零）。旋转方向上，工业仪表刻度多为顺时针递增，但也存在逆时针的情况，因此同时计算顺时针（CW）与逆时针（CCW）两个方向的占比，按仪表类型选定，默认为 CW。完整换算过程如下：

```

1  # 第一步：求三条射线的极角（数学坐标系）
2  theta_P = atan2( -(P.y - C.y), (P.x - C.x) )  # 指针射线
3  theta_Z = atan2( -(Z.y - C.y), (Z.x - C.x) )  # 零刻度射线
4  theta_F = atan2( -(F.y - C.y), (F.x - C.x) )  # 满刻度射线
5
6  # 第二步：归一化到 [0, 2*pi)，按同一方向计算扇形夹角
7  delta_total = normalize(theta_F - theta_Z)  # 总扇形角（以 CCW 为例）
8  delta_ptr   = normalize(theta_P - theta_Z)  # 指针扇形角
9
10 # 第三步：计算占比并约束到 [0,1]，换算为读数
11 ratio       = clamp(delta_ptr / delta_total, 0.0, 1.0)
12 reading_pct = ratio * 100
13 physical    = range_min + ratio * (range_max - range_min)

```

此外，系统对读数实施置信度门控：仅当圆心、指针、零刻度、满刻度四个关键点的置信度均超过各自阈值（圆心/指针 > 0.3 ，刻度 > 0.2 ）时才输出读数，否则输出“无法识别”。这一机制遵循“宁可不报、不可错报”的原则。读数超过预设告警阈值时标记为 ALARM。整个换算不含机器学习黑盒，仅涉及 atan2 与比例运算，是确定性的、可逐步验证的几何过程。

4.6 多表检测与误检过滤

工业现场常出现一个画面中多块仪表并存的情况。本项目的策略是“高召回检测 + 多级几何过滤 + 逐表独立读数”：置信度阈值降至 0.10 避免漏检；经 DIoU-NMS、二次去重与长宽比/面积过滤三道关卡剔除误检；对保留的每块仪表（默认最多 5 块）各自裁剪 ROI、各自检测关键点、各自换算读数，互不干扰。实测中，系统在“一图两块压力表”场景下能稳定地将两块表分别检出并各自给出正确读数。

4.7 昇腾推理流水线

模型推理建立在昇腾三项技术之上：OM 离线模型（经 ATC 编译，针对 NPU 算子融合优化）、ACL 运行时（设备管理与推理调度）、AIPP 硬件预处理（色彩转换、归一化由 NPU 硬件完成）。为在 NPU 推理与 30 FPS 抓帧之间维持实时性，系统采用生产者-消费者架构（图 4.3），要点如下：

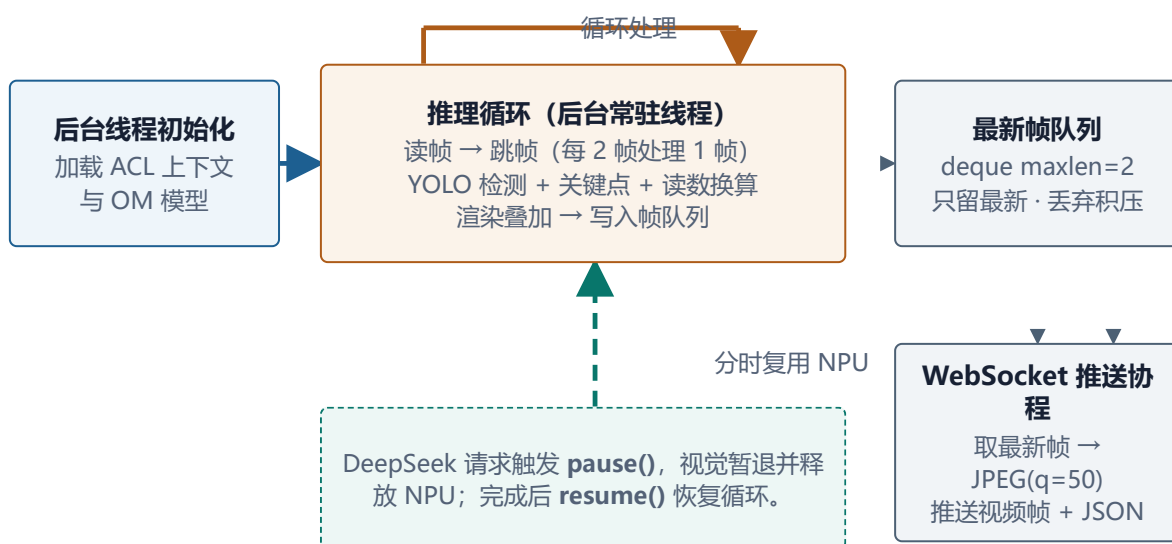


图 4.3 昇腾推理流水线工程架构图

- 后台常驻线程：服务启动即拉起独立线程，在线程内初始化 ACL 并加载模型（ACL 上下文有线程亲和性），循环执行“读帧 → 推理 → 渲染 → 入队”。
- 跳帧策略：每 2 帧处理 1 帧，被跳过的帧复用上一帧结果。仪表读数变化缓慢，视觉效果几乎无影响，有效推理负载减半。
- 帧队列（maxlen=2）：仅保留最新 1-2 帧，推送协程始终取最新帧，网络抖动不累积延迟。
- WebSocket 推送：渲染帧经 JPEG 编码（质量参数 50，平衡清晰度与带宽）后与 JSON 元数据交替推送。
- NPU 分时共享：大模型请求到来时调用 pause() 暂停视觉推理并释放 NPU 显存，推理完成后 resume() 恢复。

针对摄像头可能因震动瞬断的情况，读帧失败时自动重连（释放旧句柄、重新打开并重置参数）；连续重连 5 次仍失败则推送错误提示帧而非使服务崩溃，以支撑无人值守场景。

4.8 DeepSeek 大模型端侧部署

将巡检数据发送至云端大模型分析存在三方面问题：数据安全（巡检数据涉及生产机密）、网络可靠性（工业现场常无稳定外网）、成本与延迟。因此本项目将大模型置于机器人本体离线运行，与“全栈国产自主、断网可用”的原则一致。

模型选用 DeepSeek-R1-Distill-Qwen-1.5B（FP16）——DeepSeek-R1 蒸馏至 15 亿参数的小模型，兼具推理能力与边缘可运行的体量；推理框架采用国产 MindSpore + MindNLP，权重本地加载，全程零网络请求。部署流程与优化手段如图 4.4 所示：

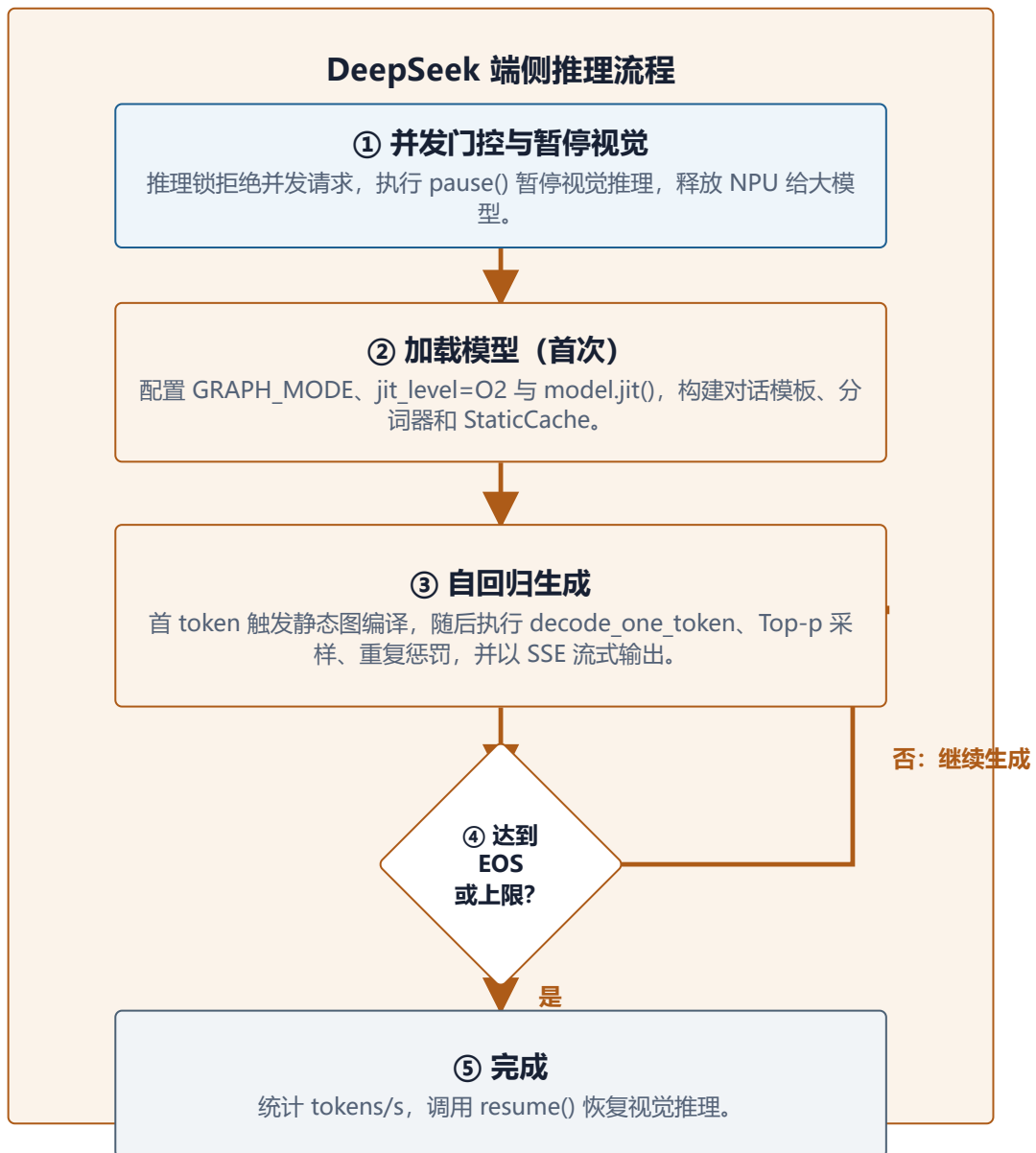


图 4.4 DeepSeek 端侧推理（JIT + StaticCache + 流式输出）流程图

- 并发门控：大模型推理独占 NPU 算力，通过推理锁拒绝并发请求，确保资源独

占。

- 与视觉分时共享 NPU：推理前暂停视觉流水线并释放显存，结束后恢复，两类任务在同一块 NPU 上共存。
- 图模式 JIT 编译：设置 `GRAPH_MODE` 与 `jit_level=02` 并调用 `model.jit()` 将整模型静态图化，生成 NPU 高效执行形式；代价是首次推理需约 140 s 图编译，之后推理速度明显提升。
- StaticCache KV 缓存：预分配固定长度（4096）的 Key/Value 缓存，避免自回归生成中重复计算历史 token。
- 采样与生成：Top-p 核采样（ $p = 0.8$ ）加重重复惩罚（1.2）抑制重复输出；采样 softmax 用 NumPy 实现（实测在昇腾开发板上更快）。
- 流式输出：生成内容经 SSE 边生成边推送，平板上文字逐步呈现。

由此，系统从“报数据”扩展到“给结论”：操作者可请求对历史读数做趋势分析，DeepSeek 基于本地数据库生成自然语言分析与异常研判报告（可导出 Markdown/txt），使机器人不仅能上报读数，还能给出分析结论。

4.9 视频流与数据服务接口

香橙派的视觉能力通过 FastAPI + Uvicorn 服务对外暴露，部署于本机 8000 端口，核心接口如表 4.1 所示。

表 4.1 视觉服务核心接口一览

类别	接口	说明
实时视频	WebSocket /ws/video	持续推送 JPEG 二进制帧与 JSON 元数据（交替发送）
状态查询	GET /api/summary	实时 FPS、延迟、读数列表（含 NORMAL/ALARM 状态）
历史数据	GET /api/history	最近 600 条读数记录
报警阈值	POST /api/thresholds	设置告警上下限（百分比）
仪表配置	GET/POST /api/gauge/configs	量程、单位、显示名等标定配置
大模型	POST /api/llm/query	DeepSeek 对话（SSE 流式输出）
分析报告	POST /api/data/report	生成趋势分析报告（调用 DeepSeek）

其中视频流是核心接口，交互时序如图 4.5 所示。建立 WebSocket 长连接后，服务端对每一帧先推一条二进制消息（已渲染检测框、关键点、读数的 JPEG 图），紧接

一条文本消息（该帧的 JSON 元数据：帧号、FPS、推理耗时、检测框、关键点坐标与各仪表读数）。图像与元数据同通道交替推送，使画面与读数一一对应，无需额外同步机制。

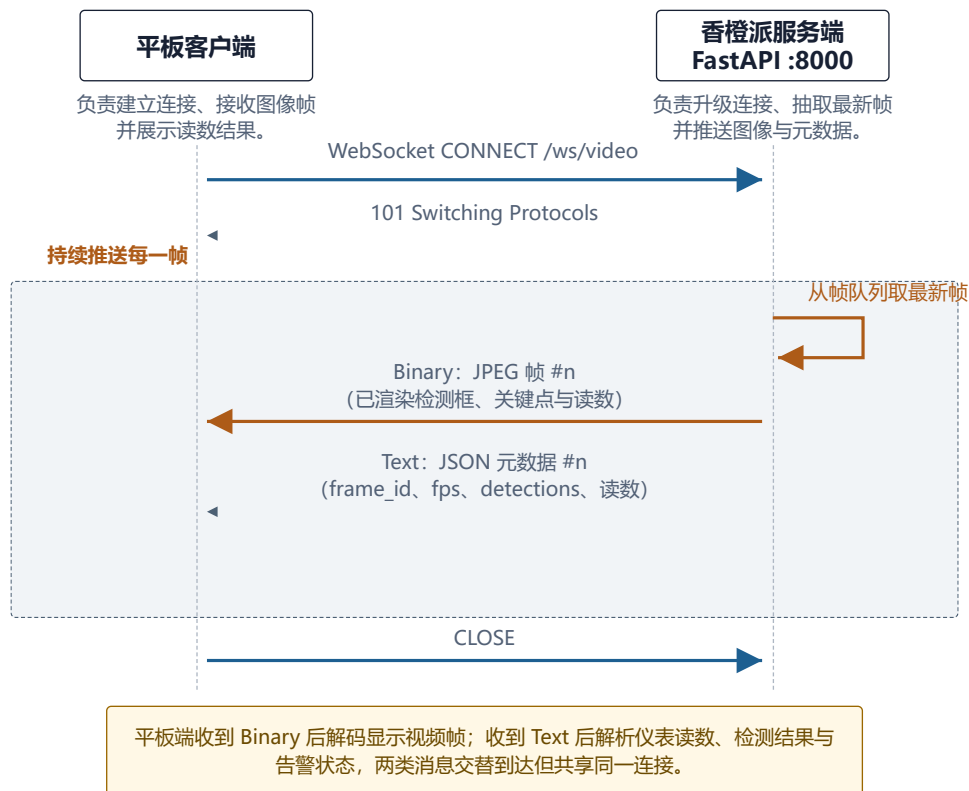


图 4.5 WebSocket 视频流交互时序图

4.10 数据采集、存储与分析

每帧识别出的有效读数均由采集模块写入本地数据库（含时间戳、仪表类型、读数、置信度、FPS 等字段），并提供按时间范围与仪表类型的统计查询，作为大模型分析报告的数据源。“采集 → 存储 → 分析 → 报告”的链路使系统能够持续积累巡检数据：读数结构化入库、可查询、可追溯，为预测性维护提供数据基础。

第五章 自主导航与运动控制

本章介绍运行在紫派（RK3566 / OpenHarmony 5.0）上的导航与运动控制子系统，涵盖激光雷达采集、SLAM 建图与定位、A* 全局规划、DWA 局部避障、全覆盖路径生成、多机协同避障与差速轮控制。与基于 ROS 导航栈的方案不同，本项目在 OpenHarmony 上以 C/C++/Python3 实现了完整闭环，仅以 LCM 作为进程间通信总线。

不依赖 ROS 并非简单更换框架，而是出于国产操作系统适配与真机可调试性的综合考虑：ROS 的 gmapping、amcl、move_base 等模块会将建图、定位、规划与参数体系绑定于境外开源生态；全部纳入自有代码后，交叉编译、板端部署与协议联调均可逐文件追溯。

5.1 子系统结构

紫派侧的设计目标是将硬件运动抽象为可被平板稳定调用的导航能力：平板只需发送 9 字节 UDP 命令，紫派内部将其转换为建图、目标点、全覆盖等控制事件，导航进程维护地图与位姿并输出速度指令。子系统由四个常驻进程构成（表 5.1），经 LCM 总线协作，运行链路如图 5.1 所示。主总线 ttl=0 仅限本机，协同避障使用独立多播通道，避免多车运行时互相污染内部信道。

表 5.1 紫派四进程概览

进程	语言	主要职责	关键输入/输出
lidar_driver	C/C++	读取 RPLIDAR 雷达，切分完整扫描帧并发布点云	发 布 HOKUYO_LIDAR
Navi	C/C++	SLAM 建图、定位、A* 规划、DWA 避障、覆盖规划与协同避障	订 阅 雷 达/里 程/控 制， 发 布 PATH、CURRENTPOSE
serial	C	差速轮速度解算、串口下发、里程估计与手动遥控	订 阅 PATH、wheel_ctrl，发 布 POSE
udp2lcm	C/Python3	UDP 5001 桥接、心跳回传、看门狗、HTTP 地图服务	订 阅 CURRENTPOSE， 发 布 ROBOT_CONTROL、wheel_ctrl

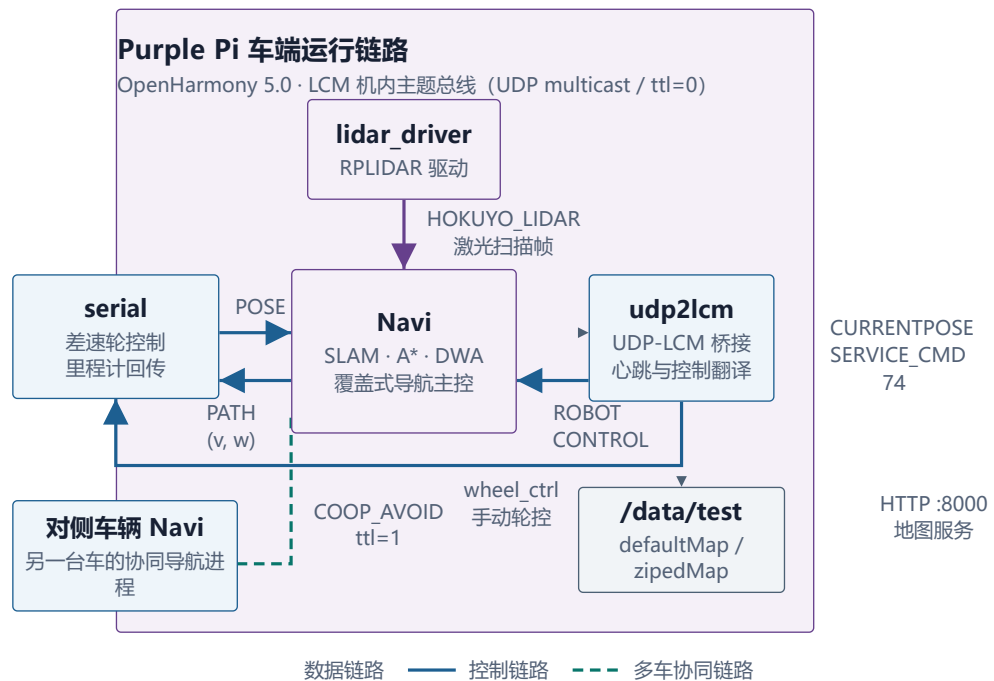


图 5.1 紫派内部运行链路图

5.2 激光雷达驱动与统一感知链路

系统采用单雷达统一感知方案：同一雷达数据同时服务于建图、定位匹配、动态障碍判断与导航避障，硬件链路简单、时间源一致。驱动基于 RPLIDAR SDK 改造，经 `/dev/ttyUSB0 (115200)` 连接雷达，采用角度回绕判定切分完整扫描帧——当扫描角度从接近 360° 回到 0° 附近时判定一圈结束，封装为 `laser_t` 消息发布至 `HOKUYO_LIDAR` 信道。扫描点缓存设置了容量上限，避免异常数据包拖垮进程；驱动还周期性输出扫描频率、点数与失败次数等统计信息，便于真机调试定位问题。

5.3 UDP-LCM 桥接与 9 字节控制协议

`udp2lcm` 是平板应用与紫派算法栈之间的边界层：平板端只面对 UDP 5001 的 9 字节协议，内部进程只面对 LCM 信道，协议要点如表 5.2 所示。

表 5.2 9 字节 UDP 协议要点

项目	约定
端口 / 包长	UDP 5001, 固定 9 字节, 大端字节序
坐标单位	1 个整数单位 = 5 cm (与地图栅格一一对应)
心跳	紫派每约 500 ms 回传一帧当前位姿
安全看门狗	约 3 s 未收到有效控制包则车端自动急停
发现机制	广播 0x06 发现包仅回响应, 不建立控制会话

命令包的 `byte0` 为命令码，其余字节按命令复用（方向、顶点序号、目标坐标或 IP 段），主要命令如表 5.3 所示。桥接层将命令解析后发布为 LCM 控制事件；心跳包则携带融合位姿与协同避障状态：`byte0` 固定为 3，`byte1/byte2` 为协同避障状态与事件，`byte3-6` 为当前 x 、 y （5 cm 单位整数），`byte7-8` 为朝向角（度）。

表 5.3 9 字节协议主要命令码

byte0	命令	紫派动作
0x6d (m)	强制重新建图	清理旧图与旧路径产物，进入建图状态
1	手动遥控	发布 <code>wheel_ctrl</code> ，用于建图阶段
2	结束建图	停轮、保存地图并加载为导航图
3 / 4	设置目标点 / 取消导航	目标点换算为米制后下发规划 / 删除目标并停车
5 / 6	加载地图 / 发现	加载默认地图（子机归零入场）/ 仅回发现响应
102-104 (f/g/h)	单车全覆盖	收集区域顶点后启动/取消全覆盖， <code>byte1</code> 携带算法号
105 (i)	子机拉主车地图	主车 IP 编码于字节中，经 <code>wget</code> 拉取压缩图
107-108 (k/l)	分布式覆盖	暂存对角点后生成本车覆盖路径并按车号执行

协议内嵌三类安全机制：车端看门狗独立于平板运行，3 s 无有效控制包即停车；发现包与普通控制包严格分离，避免广播发现误连子网内其他车辆；强制重建命令保证重新建图时清理旧产物，避免在旧图上定位与存图。此外，桥接进程在 `/data/test` 下拉起 HTTP 服务暴露地图文件——小包控制走 UDP、大文件传输走 HTTP，二者互不阻塞。

5.4 SLAM 建图与地图管理

收到强制建图命令后，Navi 先检查地图生命周期状态（建图、保存、导航中等状态下拒绝重入），随后清理旧地图与覆盖产物，重置扫描匹配器与运行时状态，进入实时增量建图：每收到一帧激光，将雷达点转换至车体坐标系，结合里程估计的预测位姿做多分辨率扫描匹配，并将结果增量融合进概率栅格地图。结束建图时，系统先保存未优化地图，再经优化流程生成默认导航图 `defaultMap.txt`，同时生成 ZMAP1 压缩图 `zipedMap.txt` 供快速拉取（压缩格式见第 8.2 节）。子车拉图时优先拉取压缩图并本地解压，校验失败或不可用时回退至普通地图。

5.5 定位与位姿更新

定位由轮控里程估计、激光扫描匹配与粒子滤波共同构成。`serial` 按当前线速度、角速度与时间差积分出连续里程估计，以约 15 ms 周期发布 POSE；激光帧进入 Navi 后与当前地图做多分辨率扫描匹配——先在粗分辨率大范围搜索可能位姿，再

逐层细化，兼顾搜索范围与实时性；粒子滤波模块用于抑制里程估计的长期漂移。融合后的定位结果以 CURRENTPOSE 发布，作为心跳回传、规划与协同避障的统一位姿来源。系统还对位姿角度做归一化处理，避免 $\pm\pi$ 附近的角度跳变。

5.6 A* 全局规划与 DWA 局部避障

单点导航采用 A* 全局规划与 DWA 局部速度规划的组合。A* 并非简单的二值通行搜索，而是在多级代价栅格上规划：静态障碍按机器人半径膨胀，激光与视觉动态障碍也参与代价计算，规划结果倾向于远离墙体与障碍而非贴边行驶。规划失败时系统区分“地图本身无路”与“临时障碍占据路线”：后者先停车、等待并重试，必要时重新规划或在多机场景下触发协同避障，避免过早终止任务。

DWA 根据当前速度与加减速约束，在速度空间内采样若干 (v, w) 候选，预测短时轨迹并按碰撞安全性、目标朝向与速度效率评分，最优速度经 PATH 信道发送给轮控进程。接近目标时收紧速度选择范围，使停车与转向更平稳。

5.7 全覆盖路径规划与多机分区执行

全覆盖分单车与多机两类。单车模式支持两种算法：zigzag 牛耕式覆盖（结合方向探测与 BFS 目标选择逐步推进）与 STC 生成树覆盖（将地图转换为机器人尺度粗栅格，生成覆盖树并按 DFS 顺序执行）。

多机矩形覆盖由平板下发两个对角点与车号触发。Navi 在当前地图上将矩形区域转换为栅格，优先采用 BCD 牛耕式单元分解生成覆盖路径——按列扫描识别连通 cell，在每个 cell 内生成往复扫描路径，并用 BFS 连接相邻 cell；若 BCD 因地图退化失败，则回退至矩形 zigzag 方案保证任务可执行。覆盖路径写入本地文件后按 robot_id 分工：0 号车执行前半段，1 号车执行后半段并逆序以降低重复覆盖。执行中提取方向变化处的转折点作为导航目标，逐段交给 A* 与 DWA；若某目标被临时障碍阻挡，则在邻近半径内寻找可通行栅格修复。

该设计将“覆盖路径生成”与“路径执行”分离：BCD/STC/zigzag 负责生成应覆盖的空间序列，A* 与 DWA 负责把序列转化为实际运动，协同避障负责多车相遇时的暂停与恢复，三层职责分明。

5.8 车间协同避障 COOP_AVOID

COOP_AVOID 是两车 Navi 之间的直连协同避障通道，使用独立多播 LCM (ttl=1, 可跨车)，不经过平板与软总线，专门处理执行层的偶发路线冲突。其工作流程为：本地 DWA 连续重试失败且判断被对方车辆阻挡时，先请求对方位姿与目标，

比较自身前瞻路线与对方位置走廊的距离；判定冲突后请求对方停车（对方保存当前目标与覆盖进度后零速度停等），本车安全通过后再请求对方恢复执行。为避免两车同时请求停车导致死锁，以 `robot_id` 做确定性仲裁：1 号车优先，0 号车停等。通道内配有 ACK 确认、约 450 ms 间隔重发与超时兜底机制，协同消息不可达时退回本地动态避障而非无限等待。协同状态经心跳字节回传平板，使“为对方停等”不被误判为掉线或卡死。

5.9 差速轮控制与串口安全

`serial` 进程订阅 `wheel_ctrl`（手动）与 `PATH`（导航），按两轮差速模型将线速度 v 与角速度 w 解算为左右轮速度百分比，经串口（115200, 8N1）下发至电机驱动板，数据包含帧头、左右轮方向/速度与异或校验。工程上做了几处安全处理：角速度限制在安全范围内，非零低速命令提升至最小有效轮速以避免“有命令但不动”；输出轮速限制在 $\pm 60\%$ 以内；串口写失败时加锁重试并重新打开串口，同时将里程估计速度冻结为 0，避免硬件异常时继续累计错误位姿。

5.10 运行状态管理与接口边界

围绕算法运行还有一层状态管理，使异常时系统行为可解释：地图生命周期保证建图前清理旧产物、存图后导航图与压缩图同时可用；导航目标在规划前先检查可通行性与动态障碍，仅满足安全条件才进入 A^* ；动态障碍采用“停车重试 $\rightarrow$ 重新规划 $\rightarrow$ 协同避障”的分级降级；覆盖任务通过转折点索引与路径文件维护进度，协同停等恢复后从中断处继续，不漏扫也不重复。车端还有两道安全兜底：UDP 看门狗在上层失联时急停，串口异常时冻结里程积分。

对上层应用而言，紫派暴露的是有限且稳定的协议边界：平板不直接调用 C++ 函数、不了解 A^* /DWA 内部结构，仅通过 9 字节 UDP 下发命令、经 HTTP 拉取地图、经心跳读取位姿与协同状态。多机协同时，软总线上的任务也由车载 agent 翻译为本机 UDP 命令后进入同一入口，C++ 导航进程保持与单车模式一致。该边界使平板 UI 与导航算法可各自独立迭代。

第六章 多机协同

多机协同以华为分布式软总线为承载：多台设备通过分布式数据对象（DDO）共享同一份任务状态，各节点监听状态变化并执行本端动作，从而把多机器人分区覆盖大区域的任务组织为去中心化的协同。本章依次介绍协同场景、软总线提供的能力、共享任务黑板、设备互信、车载 agent，以及双车分区覆盖与协同避障。

6.1 协同场景与设计选择

在大规模厂区巡检中，单台机器人完成全区域覆盖耗时过长，需要多机协同：一台主车先行建图，随后各车从同一起点出发，各自领取一块子区域执行全覆盖；平板作为总控，负责触发建图、划分区域并监看所有车辆的实时位姿与进度。该场景要解决三个问题：多车如何共用同一张地图与坐标系、如何把大区域均衡分配给多车、路线交叠时如何避让。

在鸿蒙生态中，一种常见的跨设备协作方式是远程拉起：平板通过 `startAbility` 拉起车端页面，用参数传递一次性数据，页面执行完即销毁。本项目没有采用这种方式，原因是：（1）每次操作都是“拉起—传参—销毁”，无法维持持续的共享状态，不能实时反映各车进展；（2）Ability 本质是 UI 组件，被当作无状态函数反复拉起销毁，稳定性差；（3）协同逻辑分散在各次拉起的参数中，难以维护。本项目改用基于 DDO 的共享状态方式：各设备共持一份任务状态，靠状态变化驱动协同。

6.2 分布式软总线的三层能力

分布式软总线是鸿蒙操作系统的底层能力，其作用是在可信网络内把多台鸿蒙设备连接为统一的协作环境，对上层屏蔽 Wi-Fi、蓝牙等底层网络差异，使设备发现、连接与数据传输具备统一接口。对本项目而言，软总线提供了三层可直接使用的能力（图 6.1）：

- 设备发现与传输：同一网络内自动发现其他鸿蒙设备，认证后在设备间透明传输数据。本项目通过 `distributedDeviceManager` 使用该层能力。
- 共享对象自动同步（DDO）：多台设备用同一 `sessionId` 加入同一个共享数据对象后，任何一端修改对象，变更都会被软总线自动实时同步到其余各端，各端通过 `on('change')` 回调感知。编程时操作该对象与操作本地对象无异。
- 设备互信约束：DDO 同步有两个前置条件——各端应用使用相同 `bundleName`

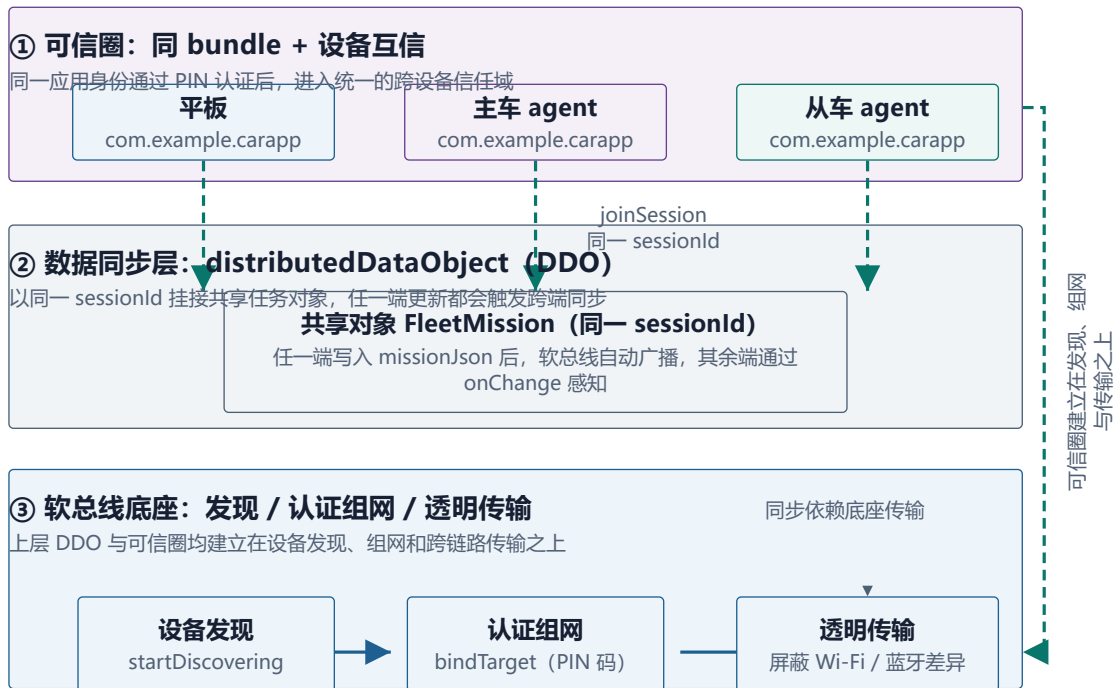


图 6.1 华为分布式软总线原理示意图

(本项目统一为 `com.example.carapp`)，且各设备已通过 PIN 码认证组成可信环。

这些能力是鸿蒙作为分布式操作系统的原生能力，不依赖第三方中间件，也无需自建消息服务器。若用传统的“Android/Linux + 自建 socket/MQTT 服务器”方案，实现等价的多设备状态同步需自行搭建发现、认证与广播设施，复杂度更高。直接复用软总线既简化了实现，也使协同能力建立在国产生态之上。

6.3 FleetMission 共享任务黑板

本项目把 DDO 能力具体实现为跨设备共享的任务黑板 FleetMission：平板与每台车的常驻 agent 加入同一会话，共持同一份任务状态，结构如图 6.2 所示。黑板的主要字段包括：任务阶段 `phase` (`idle/scanning/dividing/covering/done`)；坐标系元数据 `frame` (原点、0.05 m/格分辨率、朝向定义)，是多车共用同一坐标系的前提；地图来源 `map` (主车地图地址)；大区域边界 `area`；区域分配 `assignments` (每车负责的对角矩形与主从身份)；各车运行态 `robots` (位姿、朝向、覆盖进度、在线状态)。

基于该黑板，多机协同由共享状态驱动：平板发出建图命令后，主车进入 `scanning` 阶段；主车建图完成后把地图地址与区域边界写回黑板，阶段转为 `dividing`；平板在共享地图上划分子区域并写入 `assignments`，阶段转为 `covering`；各车 agent 监听黑板变化，读取属于本车的分配区域并翻译为本机 UDP 命令启动覆盖；覆盖过程中

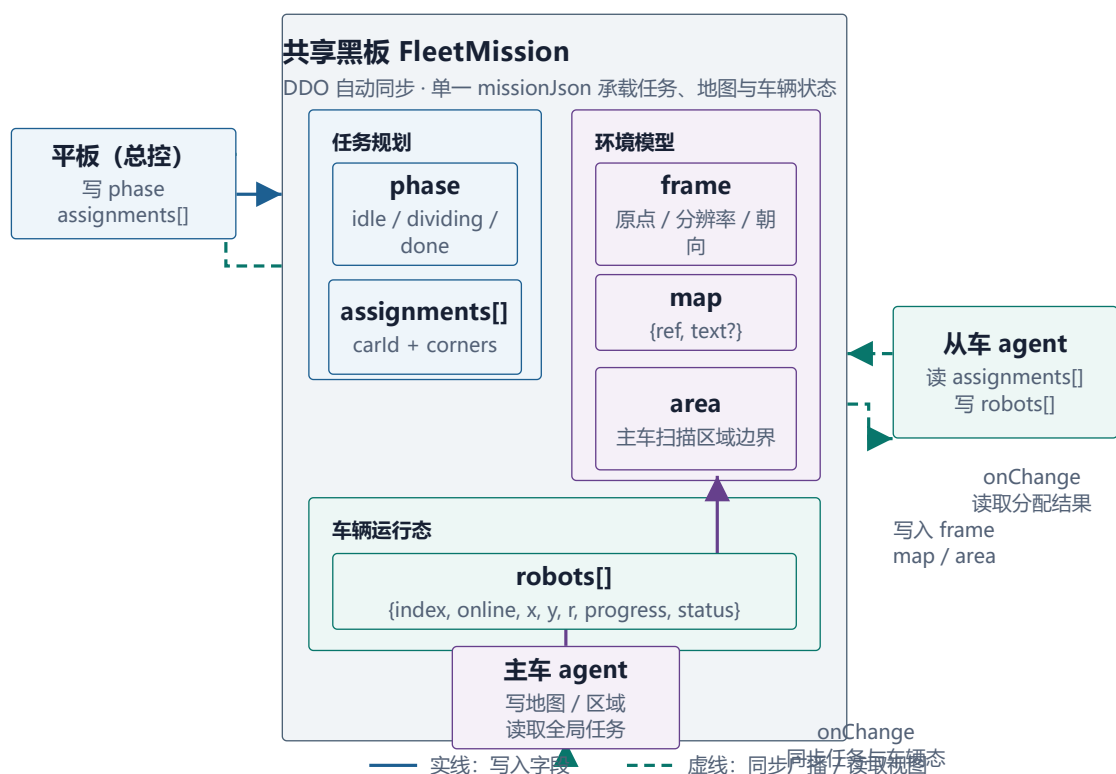


图 6.2 共享黑板 FleetMission 模型图

各车持续把位姿与进度写回 `robots`，平板据此刷新全局态势。每个节点只需读取任务状态、写入本机进展、响应状态变化，无需远程调用，也无需维护额外的命令会话。

工程实现上，整块黑板序列化为一个 **JSON** 字符串写入 **DDO** 的单一字段同步，而非拆成多个字段——这样避免了多字段异步到达产生的中间不一致状态，变更回调也只需处理一份完整快照。

6.4 设备互信与 PIN 认证

DDO 能同步的前提是各设备先建立互信、组成可信环，时序如图 6.3 所示。流程为：平板以 **bundleName** 创建设备管理器并开启发现，软总线在同一网络内搜索其他鸿蒙设备；平板对选中的车端发起绑定（**bindType=1**，无账号 PIN 码认证），车端弹出系统 PIN 弹窗，配网阶段通过 HDMI 显示器确认 PIN 码；认证通过后两端建立持久互信（跨重启有效），此后只要使用相同 **bundle** 与同一 **sessionId** 加入 DDO，即可共享黑板。项目还提供解绑能力，用于绑定异常时重置关系。

工程上有一个值得注意的细节：DDO 协同需要 `DISTRIBUTED_DATASYNC` 权限，车端无界面 agent 无法弹框申请，必须预授权；且初始化时须先申请权限、再创建 DDO 对象，否则 `joinSession` 会被静默跳过，表现为“入会假成功而实际未连通”。

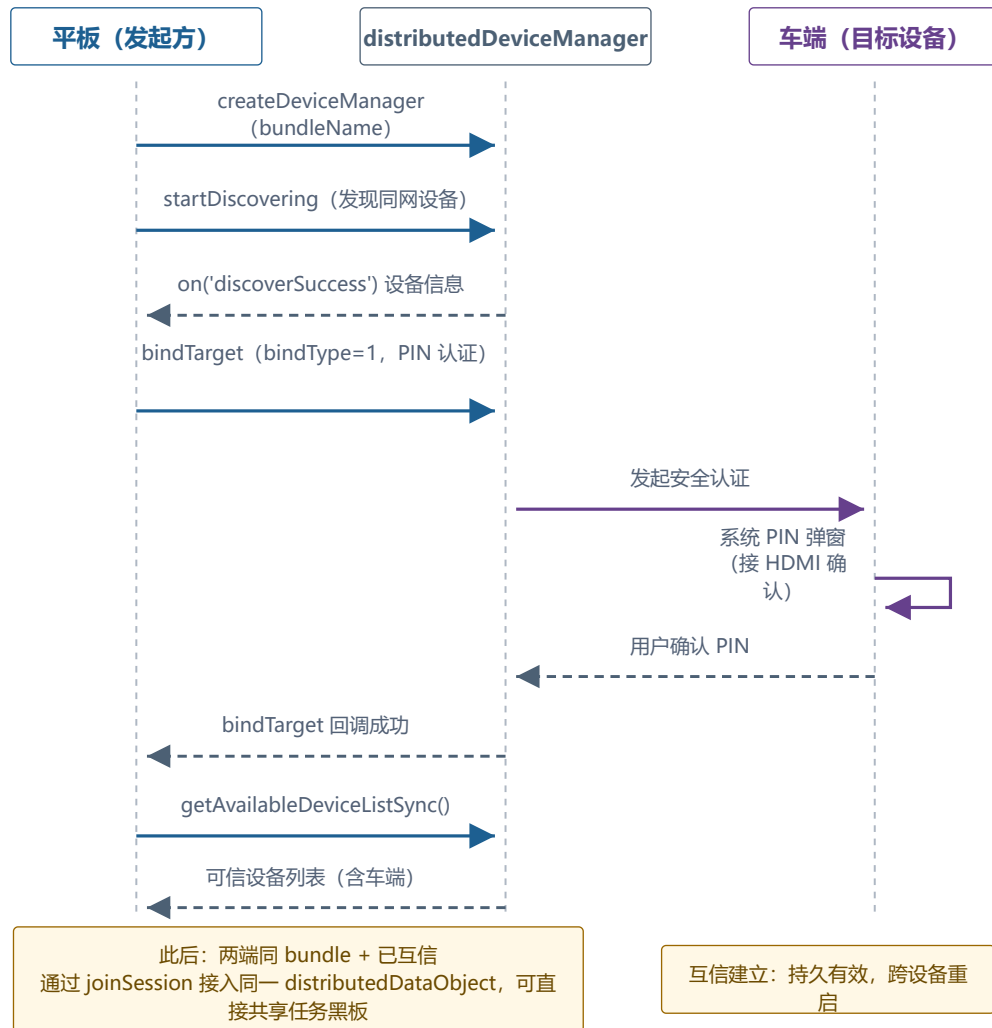


图 6.3 设备互信（distributedDeviceManager + PIN）时序图

6.5 车载无界面 agent

黑板上的协同决策最终要转换为紫派能理解的 9 字节 UDP 命令，承担这一翻译职责的是常驻车上的无界面 ArkTS agent。它做双向桥接：一方面监听黑板变更，把与本车相关的决策翻译为本机 UDP 命令下发给 udp2lcm；另一方面接收本车回传的位姿心跳，经节流后写回黑板的 robots 字段供平板监看。

agent 的命令下发逻辑可概括为“期望状态与实际状态对齐”：黑板描述期望状态（本车应覆盖的区域），agent 对比本车当前状态，计算出为达成期望所需的命令序列；其关键特性是幂等——只要与本车相关的黑板内容没有变化，就不重复下发命令。具体流程如图 6.4 所示：先从整块黑板快照中抽取与本车相关的切片（阶段、分配区域、主车地址等）并计算签名，签名未变则直接返回；进入 covering 阶段且本车有分配区域时，区分主从身份——从车先经 cmd105 从主车拉取地图，再经 cmd5 加载地图并将位姿归零（从车与主车同一物理起点出发，归零后即为地图原点），主车则使用自建的地图与定位；随后两者均下发两个对角点命令，触发紫派生成本车覆盖路径并执行。由于紫派采用“先暂存后使用”的命令机制，agent 在命令对之间保留间隔（拉图后预留 15 s 供传输完成），避免乱序。

实现上，agent 的运行核心（绑定 UDP、加入软总线、命令生成、状态回写）与 Ability 外壳分离：当前以 UIAbility 外壳托管核心逻辑，后续以系统应用形式部署时只需更换为 ServiceExtensionAbility 外壳，核心无需修改。此外，agent 仅在本车确有任务时才作为 UDP 客户端保活，空闲与建图期间保持静默，避免与平板直连争抢车端 5001 端口。

6.6 双车分区覆盖与协同避障

多车覆盖的正确做法不是把一条路径对半分、两车从两端相向而行（那样两车必在中点相遇、冲突频发），而是空间分区：在同一张地图上把大区域划分为空间上分离的子矩形，每车负责一块，在本车矩形内独立执行 BCD 全覆盖（见第 5.7 节）。空间分离使两车很少相遇，把协同避障从主力机制降为偶发兜底，提升了多车覆盖的稳健性。

即便分了区，两车在子区域交界带仍可能路线交叠，此时由 COOP_AVOID 车间直连通道处理（机制详见第 5.8 节）。该通道在两车的 Navi 之间闭环，使用独立多播 LCM，不经过平板与软总线，对上层完全透明：触发后先交换位姿判断是否真的冲突，再协商停车与恢复，以 robot_id 仲裁避免对称死锁，并以 ACK、重发与超时回退保证不可靠多播下的可靠协商，交互时序如图 6.5 所示。

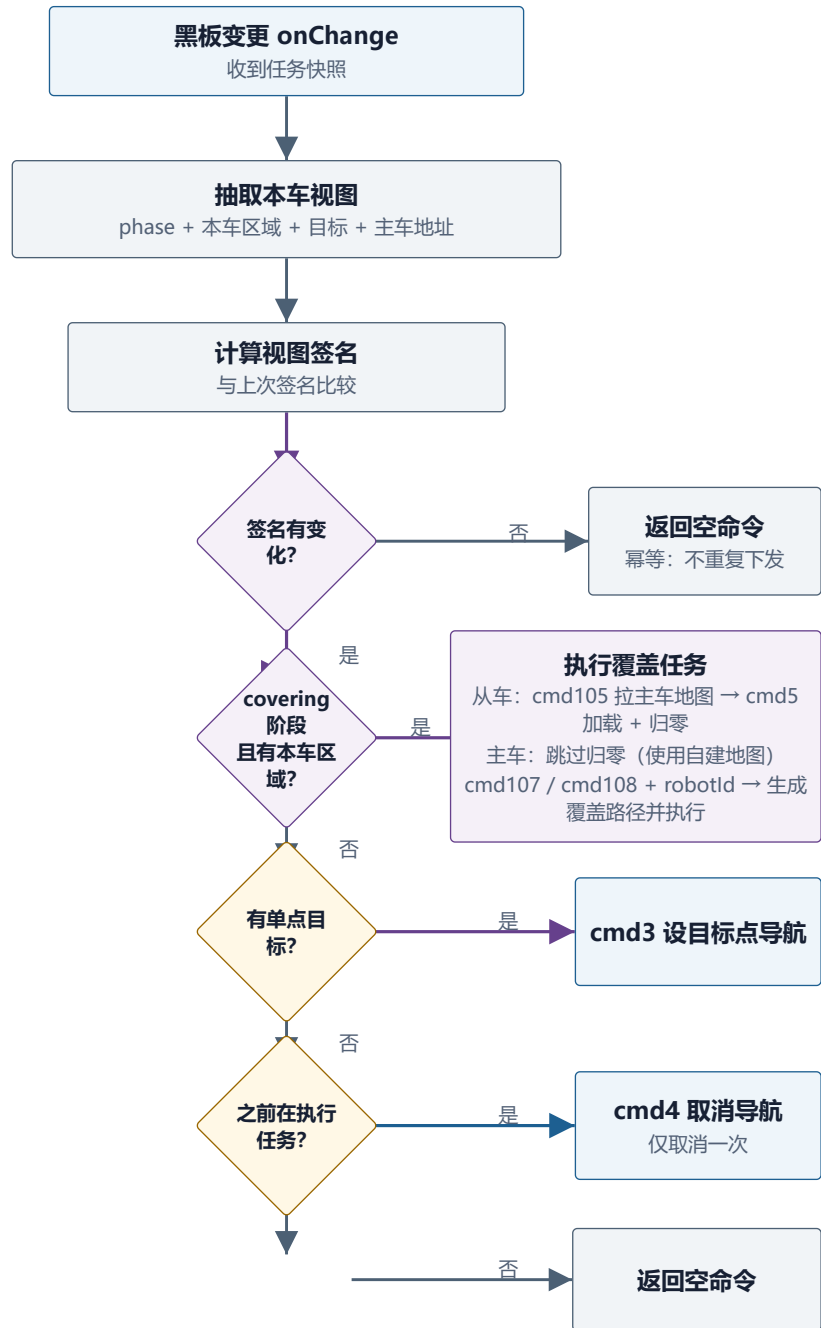


图 6.4 车载 agent 工作流程

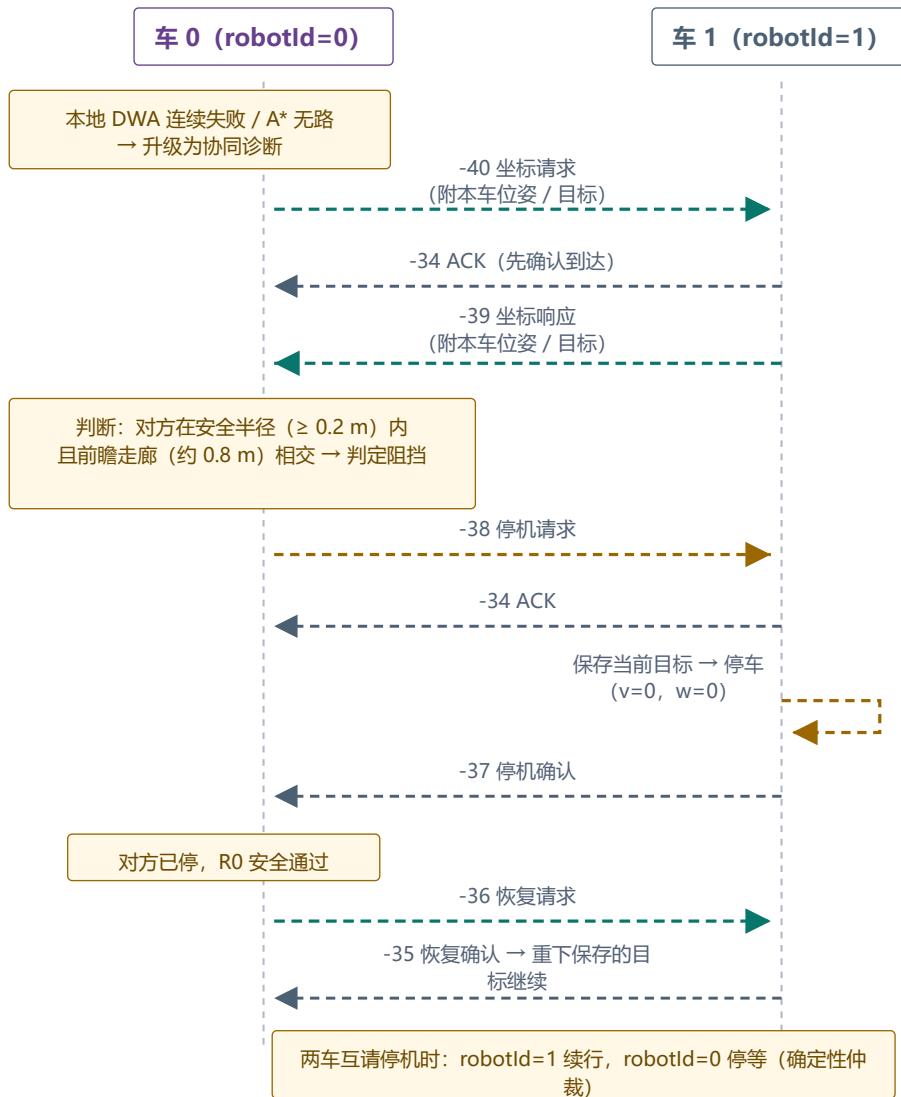


图 6.5 双车协同避障 (COOP_AVOID) 时序图

综上，多机协同以分布式软总线为承载，用共享任务黑板替代远程拉起传参，以 PIN 认证互信保证同步安全，以车载 agent 把黑板决策幂等地翻译为本机命令，并以空间分区与 COOP_AVOID 实现双车协同覆盖与避障。

第七章 平板控制应用

本章介绍系统的人机交互终端——运行在鸿蒙平板上、采用国产 ArkTS 语言开发的控制应用。该应用承担建图触发、运动遥控、目标与区域下发、地图与识别结果可视化，以及多机协同总控等职责，从语言（ArkTS）、UI 框架（ArkUI）到分布式能力（软总线）均基于鸿蒙原生生态构建。

7.1 应用总体架构

应用采用 ArkTS 与 ArkUI 声明式框架开发，并按页面、组件、服务、模型四层组织（表 7.1）。关键原则是：UI 层不直接操作 socket、不直接拼接协议，所有与车辆及视觉服务的通信均统一由服务层处理（例如 RobotTransport 是全应用唯一的 UDP socket 持有者），UI 层仅调用服务层的语义化方法。这种做法避免了“各页面各自监听 socket”导致的监听泄漏与资源抢占，也保证了协议实现的一致性。

表 7.1 应用分层架构

层次	模块	职责
页面层	HomePage / ControlPage / VisionPage / SetIPPage / DeviceTrustPage	UI 渲染与交互
组件层	MapCanvas / Joystick / VideoView / ReadingPanel / DeviceList	可复用 UI 组件
服务层	RobotTransport / MapService / VisionService / FleetMissionService	通信、地图、视觉、协同
模型层	protocol / mission / mapContour / vision / geometry	协议编解码、数据结构、坐标几何

7.2 统一传输层与多车保活

第 5.3 节所述协议包含一条安全约定：应用必须至少每秒向车辆发送一帧指令，否则车辆 3 秒后急停。这意味着应用必须持续保活；当平板管理多台车辆时，需为每台车各维护一条保活循环。RobotTransport 作为全应用唯一的 UDP 传输单例，统一持有 socket、统一收发 9 字节协议，并用 Map<ip, HeartbeatLoop> 为每个目标 IP 维护独立的保活定时器；消息回调只注册一次，按订阅者分发解析后的心跳，杜绝监听泄漏。

在此之上，每台车的连接状态由“是否保活”与“心跳新鲜度”派生为一个状态机（图 7.1）：已发现（广播发现车辆但未启动保活）、连接中（已保活但尚未收到心

跳)、已连接(心跳新鲜,链路健康)、已断开(曾经连接,心跳超时 2 秒,UI 提示连接丢失)。UI 据此精确反映每台车的链路健康度。

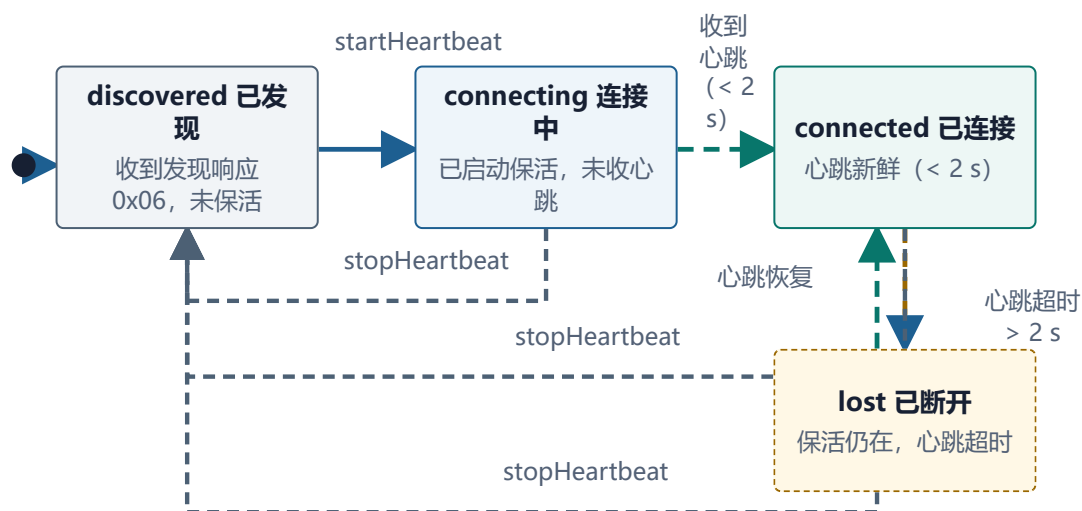


图 7.1 应用与车辆连接状态机

传输层还内置了线控日志：显式命令的发送始终打印命令名与 9 字节十六进制内容，高频保活与心跳默认静默、仅在内容变化时打印。调试者可以直接核对“实际发出去的字节是否正确”，这在多设备联调中非常实用。

7.3 地图渲染与坐标换算

地图可视化要把紫派构建的、规模可达 1800×1800 的栅格地图正确渲染到屏幕上，并支持点选目标与框选区域，核心是三套坐标系的互逆换算：真实世界坐标（紫派建图坐标，5 cm/格，与地图栅格一一对应）、屏幕地图坐标（对地图做障碍包围盒裁剪与正方形化后的可缩放显示坐标）、触控像素坐标（用户手指点选位置）。下发目标点时，屏幕像素逐步换算回世界坐标；渲染位姿时，心跳中的世界坐标逐步换算至屏幕像素，两个方向必须严格互逆。

地图文件解析有两个实现要点。其一，首行 7 个字段中，行列数须取第 3、4 个字段（height/width），末尾的 x0/y0 是最小角的世界坐标偏移（真机上常为负值），取错会导致地图整体错位。其二，压缩地图（ZMAP1，见第 10.3.3 节）把 64 个栅格打包为一个 64 位无符号整数，而 ArkTS 普通 number 只有 53 位精度，无法精确表示——应用端须使用 BigInt 读取，再按位移运算逐位还原栅格。

7.4 视频流与读数展示

VisionService 通过 WebSocket 连接香橙派的 /ws/video 接口，按第 4.9 节所述方式接收“JPEG 帧 + JSON 元数据”的交替推送：二进制帧解码渲染至视频组件，JSON 解析出读数、置信度与告警状态更新至读数面板。视觉页与控制页通过设备发现联动：应用广播发现包时，车辆回复 0x06、视觉源回复 0x07，应用据此自动区分两类设备并配置视觉服务地址，无需手工填写。视觉页还实现了断线 3 秒自动重连与 ALARM 状态闪烁提示。

7.5 任务编排与交互设计

作为总控终端，应用把各子系统能力编排为完整操作流：

- 单车流程：发现并连接车辆 → 开始建图 → 摇杆遥控遍历环境 → 结束建图 → 拉取并渲染地图 → 点选目标点或框选区域启动全覆盖 → 实时监看位姿与覆盖轨迹 → 视觉页查看实时读数 → 请求生成分析报告。
- 多车流程：在单车基础上，经 FleetMissionService 加入软总线、建立设备互信、共享任务黑板，在共享地图上为多车划分子区域并监看全局态势（见第六章）。

交互上，应用提供摇杆遥控（长按持续发送、松手即停）、地图触控选点与框选、设备列表与互信管理、IP 配置等组件，操作者在一个应用内即可完成“控车—看图—读表—分析”的全流程。

第八章 性能优化

本章从数据通路与端到端耗时的角度，说明系统的性能瓶颈定位与已落地的优化措施，并汇总关键性能指标。

8.1 数据通路与瓶颈定位

系统的数据流可归为四类通路（表 8.1）。控制与心跳是 9 字节小包，无瓶颈；主要优化对象是数据量大的地图通路——地图文件体积大、传输频次低但等待时间明显，直接影响“建图完成到地图可用”的端到端体验。早期实现中地图按行经 LCM 逐条上送，一张 $20\text{ m} \times 20\text{ m}$ 地图仅人为延迟即需约 20 秒；车与车之间拉图则使用单线程 HTTP 整文件传输。这两处是地图链路的主要延迟来源。

表 8.1 四类数据通路及其瓶颈特征

通路	载体	用途	数据量	瓶颈特征
App ↔ 紫派	UDP 5001, 9 字节	控制/心跳	极小	无瓶颈
紫派 → App	HTTP 8000	地图上送	大	整文件传输
香橙派 → App	WebSocket	视频流	中	编码/带宽
App 紫派内部	LCM	雷达/位姿/轮控	中（高频）	激光编码冗余

8.2 地图位打包压缩（ZMAP1）

占据栅格地图本质上是二值图，每格非障碍即空旷，理论只需 1 bit；而密排文本格式用 1 字节存储一个 0/1 值，信息密度仅 1/8。ZMAP1 压缩据此设计：每个栅格压为 1 bit（障碍 =1、空旷 =0），每 64 格按位打包为一个 64 位无符号整数，逐行写出，地图体积降至约 1/8。字以十进制文本形式写出以避免二进制端序问题；解压时按行读取各字，用位移运算逐位还原栅格。

获取流程采用“压缩图优先”策略：平板或子车优先拉取 zipedMap.txt，校验并本地解压；压缩图不存在、校验失败或下载失败时回退拉取普通地图。实测该方案使地图传输数据量明显下降，对 1800×1800 量级的地图尤为明显。

8.3 推理流水线提速

视觉链路的实时性依赖第 4.7 节所述的流水线工程，从性能视角汇总如下：AIPP 硬件预处理省去 CPU 浮点开销；每 2 帧处理 1 帧使有效推理负载减半；帧队列仅保留最新帧，延迟不累积；预处理采用最近邻插值，速度更快且精度损失可忽略；大模型推理采用 MindSpore 图模式 JIT 与 StaticCache KV 缓存提速。上述措施叠加后，“检测 + 关键点 + 读数 + 编码”流水线稳定运行在约 29 FPS。

8.4 端到端性能指标

系统关键性能指标汇总如表 8.2 所示。视觉单帧端到端约 34 ms（约 29 FPS），控制心跳 500 ms 一帧，地图压缩比约 8 倍，可满足巡检场景的实时性要求。

表 8.2 端到端性能指标汇总

指标	数值	说明
仪表检测（YOLOv5s）	~21 ms	昇腾 OM 推理
关键点检测	~8 ms	Simple Baselines OM 推理
预/后处理 + 编码	~5 ms	含 JPEG 编码（quality=50）
视觉端到端	~34 ms（约 29 FPS）	单帧全链路
控制心跳周期	500 ms	紫派 → App 位姿心跳
保活/急停超时	3 s	无指令即急停
地图分辨率	0.05 m/格	与 UDP 整数坐标 1:1
地图压缩比	~8×（位打包）	ZMAP1 相对密排文本
SLAM 关键帧阈值	0.35 m / 30°	位移或转角触发新关键帧
差速底盘满速	0.60 m/s	FULLSPEED

第九章 系统运行实测

本章给出系统在真实软硬件平台上的运行结果。所有结果均基于自主搭建的实物小车（紫派 RK3566/OpenHarmony 5.0、香橙派昇腾 310B、RPLIDAR 激光雷达、差速底盘、USB 摄像头）与鸿蒙平板控制端实测得出。

9.1 自主建图与导航实测

在室内场景中，操作者通过平板发送强制建图命令，紫派进入建图状态，遥控遍历环境过程中地图实时增量构建；结束建图后生成导航图与压缩图。实测表明，强制重建语义能有效避免沿用旧图：每次建图前系统清理旧地图、旧路径与覆盖产物，新地图从干净状态开始构建；地图坐标原点与 5 cm 栅格单位与平板端解析一致，压缩图优先拉取也使地图获取等待明显缩短。平板端启动建图的实测界面如图 9.1 所示。



图 9.1 平板端启动自主建图

导航实测中，操作者在地图上点选目标点，紫派以 A* 规划全局路径、DWA 输出实时速度，机器人沿规划路线自主行驶至目标点，心跳位姿持续回传，平板同步显示当前位置与轨迹。当路径被临时障碍阻塞时，系统先停车并重试、必要时重新规划，而非盲目继续前进，具备可解释的失败降级能力。平板端导航与巡检控制界面如图 9.2 所示。

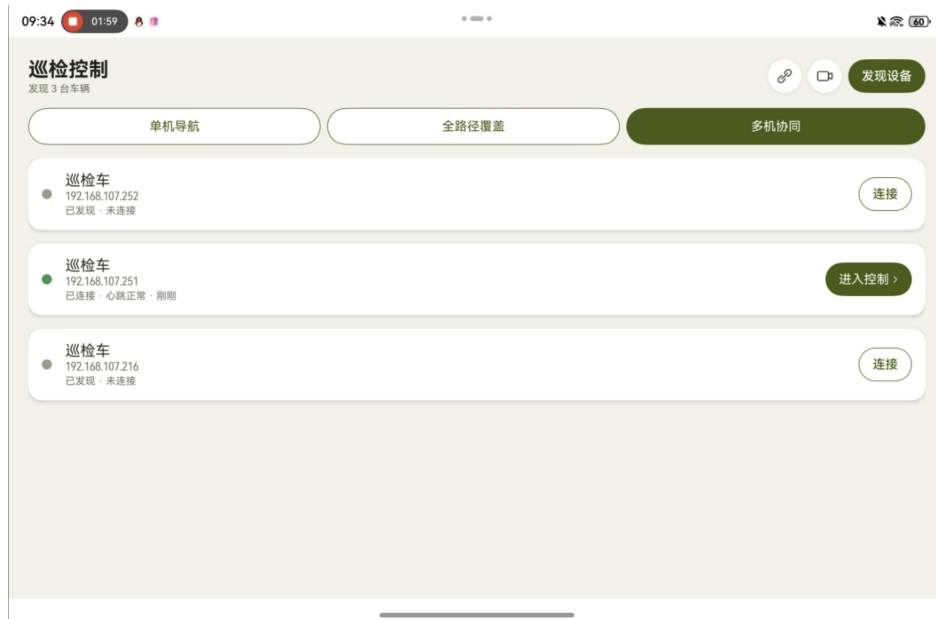


图 9.2 平板端导航与巡检控制界面

9.2 全覆盖遍历实测

单车模式下，平板下发区域顶点后启动覆盖，zigzag 与 STC 两种算法均能生成并执行完整覆盖路径。多机矩形覆盖模式下，平板向每台车下发对角点与车号，各车在本车矩形内用 BCD 生成覆盖路径并分工执行，交界处的偶发冲突由 COOP_AVOID 协商处理。实测中，该方式能把大区域拆分为多车可独立执行的覆盖任务。机器人执行全覆盖巡检的实景如图 9.3 所示。

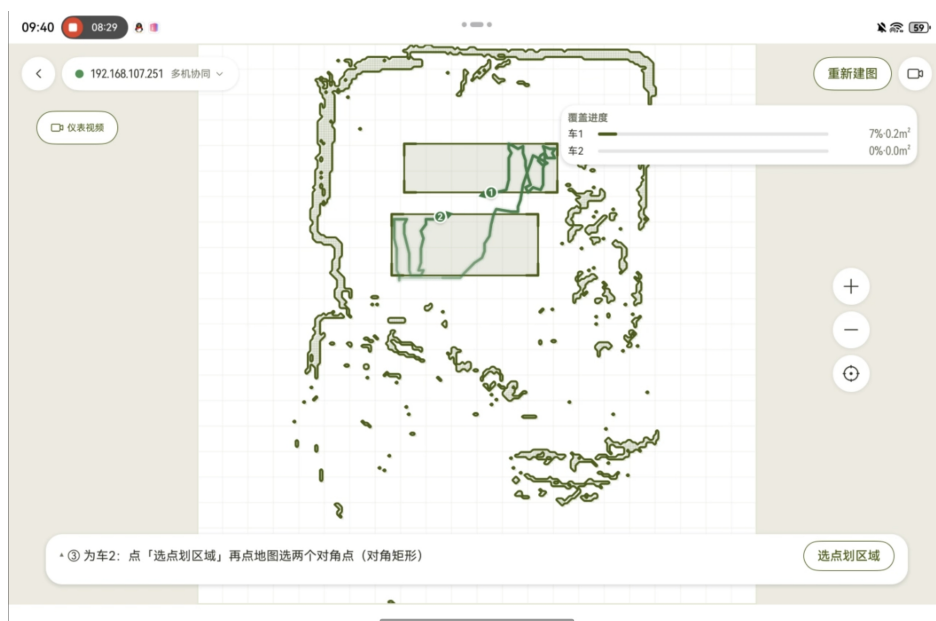


图 9.3 机器人全覆盖巡检实景

9.3 仪表识别精度与实时性

视觉子系统在昇腾 NPU 上完成了“检测 → 关键点 → 读数”全链路实测，并录制了实时推理视频作为运行证据。识别与读数实测效果如图 9.4与图 9.5所示，要点如下：

- 检测准确性：YOLOv5s 在自采压力表数据集上训练，能稳定检出画面中的仪表；配合多级过滤，跨表大框与重复框被有效剔除。
- 多表同时识别：一图两块压力表场景下，两块表被分别检出并各自给出正确读数，互不干扰。
- 读数准确性：关键点定位准确，几何换算得出的指针百分比与人工读数基本一致；置信度门控保证了“宁可不报、不可错报”。
- 越限告警：读数超过预设阈值即标记 ALARM，平板端触发告警提示。

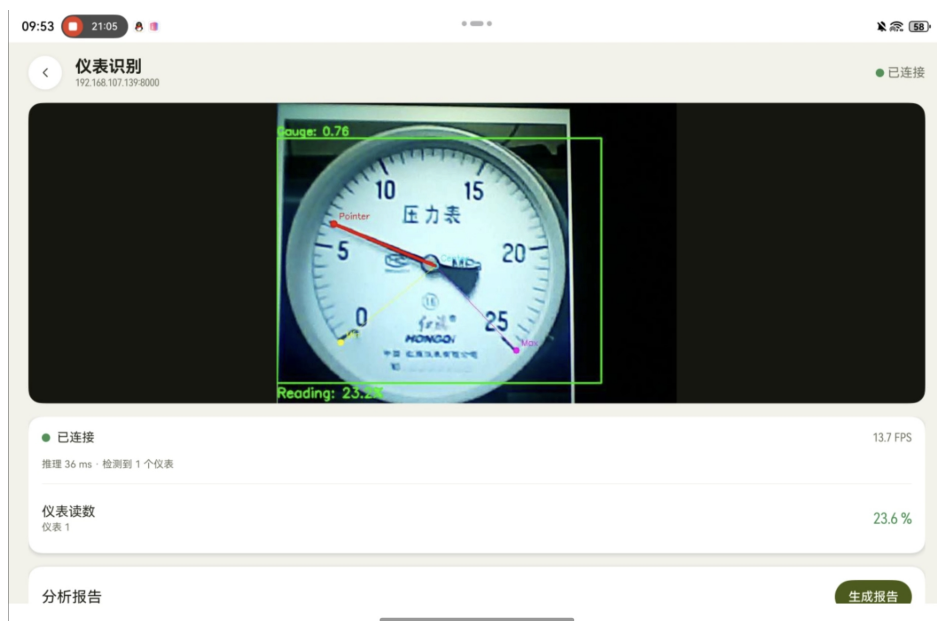


图 9.4 工业仪表识别与读数实测



图 9.5 多块仪表识别与读数结果示例

实时性方面，实测 YOLO 检测约 21 ms、关键点检测约 8 ms、后处理与编码约 5 ms，单帧端到端约 34 ms（约 29 FPS）。配合跳帧与“仅保留最新帧”的队列机制，平板显示画面流畅、延迟低，可满足巡检实时读表需求，且全部在国产昇腾边缘算力上完成，数据不出本体。

9.4 DeepSeek 端侧分析实测

操作者在平板请求“生成分析报告”后，香橙派暂停视觉推理以释放 NPU，调用端侧 DeepSeek-R1-Distill-Qwen-1.5B 对本地历史读数进行分析。实测表明：（1）模型本地加载，全程零网络请求，断网可用；（2）回复经 SSE 流式推送，文字逐步呈现；（3）能基于历史读数生成趋势分析与异常研判报告，支持导出为 Markdown 或纯文本；（4）通过“暂停视觉 → 大模型推理 → 恢复视觉”的分时调度，两类任务在同一块 NPU 上共存。需要说明的是，端侧大模型首次推理需约 140 s 的 JIT 图编译（一次性代价），编译完成后后续推理速度明显提升。

9.5 多机协同实测

多机协同实测覆盖“互信建立 → 分区下发 → 拉图加载 → 分区覆盖”的完整链路：平板与两车通过 PIN 认证建立互信（图 9.6），在共享地图上为每台车划分矩形区域；车载 agent 监听到与本车相关的任务后翻译为本机命令；从车经 HTTP 拉取主车压缩图并本地解压加载，随后各车在本车分区内生成并执行覆盖路径，平板实时显示两车位姿、朝向与覆盖进度。两车路线交汇时，COOP_AVOID 通道完成位姿交换、协商停车与恢复，心跳状态回显使平板能区分正常运行、协同停等与本地兜底。

综合实测结果，系统实现了“自主建图导航 → 全覆盖遍历 → 实时仪表识别 → 端侧大模型分析 → 多机分区协同”的完整功能流程，全部核心功能运行在国产软硬件平台之上。



图 9.6 多机协同设备互信（PIN 认证）实测

第十章 创新点

本章在前述各章的基础上归纳系统的技术创新，分为五个维度：国产化与自主可控、异构架构与通信协议、自主导航算法链、多机协同、视觉与端侧大模型结合。表 10.1 以矩阵形式概览，后续各节分别展开。

10.1 创新点总览

表 10.1 创新点总览矩阵

维度	创新内容	技术价值
国产化与自主可控	操作系统、AI 算力、开发语言全栈国产化	满足信创自主可控要求，降低外部依赖风险
	SLAM、规划、覆盖、轮控等核心算法自主实现	算法资产可审查、可修改、可迁移
	训练数据自行采集标注，模型基于公开基线改造	模型可解释、可重训
	DeepSeek 端侧离线运行（国产模型 + 框架 + NPU）	数据不出本体，断网可用
异构架构与通信协议	视觉与导航物理解耦至双计算单元	算力专用化、故障隔离、独立迭代
	四层通信方式分层设计	各层职责清晰，互不干扰
	9 字节定长控制协议	跨语言解析一致，嵌入式场景适配
	ZMAP1 位打包压缩地图（约 8×）	大块地图传输显著提速
自主导航算法链	多分辨率扫描匹配 SLAM	国产 OS 上无外部库依赖的自主建图
	KLD 自适应粒子滤波定位	粒子数自适应，抑制长期漂移
	A* 代价分层规划 + DWA 局部避障	多传感器融合避障，路径带安全余量
	BCD/STC/zigzag 全覆盖（含失败回退）	兼顾覆盖完整性与工程可执行性
多机协同与软总线	DDO 共享任务黑板	替代远程拉起，实现状态持久共享

续表 10.1

维度	创新内容	技术价值
	幂等的车载协同 agent 空间分区 + COOP_AVOID 直连兜底	状态不变则不重复下发命令 覆盖稳健，避障与上层解耦
视觉与 端侧大 模型结合	检测 + 关键点 + 几何换算 两段式读数 视觉推理与大模型分时共 享 NPU JIT 图模式 + StaticCache 端 侧大模型优化	读数过程可解释、可标定、可复现 同一块 NPU 上推理与分析共存 边缘大模型工程化落地

10.2 国产化与自主可控

在关键技术自主可控需求日益迫切的背景下，本系统从四个层面实现了完整的国产化路径。

操作系统层面，运动主控与交互终端均运行开源鸿蒙（OpenHarmony / HarmonyOS），完整的机器人栈部署于该平台，并利用鸿蒙原生的分布式软总线实现多机协同——这是传统单机操作系统不具备的能力，也验证了国产操作系统承载复杂机器人应用的可行性。

AI 算力层面，视觉推理与端侧大模型均运行于华为昇腾 310B NPU，模型经 ATC 编译为原生 OM 格式、由 ACL 运行时调度、AIPP 硬件完成预处理，完全不依赖境外 GPU 产品。

算法层面，不使用 ROS 及任何境外机器人中间件，SLAM、规划、覆盖与轮控等核心算法全部自主实现，内部通信采用可在 OpenHarmony 上零依赖移植的轻量级 LCM——全部核心代码可审查、可修改、可优化，不存在黑盒依赖。

数据与模型层面，视觉训练数据自行采集标注；DeepSeek-R1-Distill-Qwen-1.5B 采用国产 MindSpore 框架在昇腾 NPU 上完全离线运行，数据全程不出设备本体。整车硬件结构与供电系统也均自主搭建，从硬件到软件实现完整的自主可控。

10.3 异构架构与通信协议设计

10.3.1 双单元物理理解耦架构

视觉感知与运动导航分别部署于两块独立计算单元：香橙派负责视觉推理，紫派负责运动控制。二者不共享进程与算力，通过车内网络交换控制指令与运行状态。香橙派识别到仪表后向紫派下发减速指令，读数完成或目标离开后下发恢复指令；紫派执行后回传当前行进状态。该交互不改变两块板卡的部署边界，系统仍具有算力专用化（NPU 专注 AI 推理、CPU 专注实时控制）、故障隔离（一侧异常不影响另一侧）与独立演进（模型与算法各自迭代）三方面收益，如图 10.1 所示。

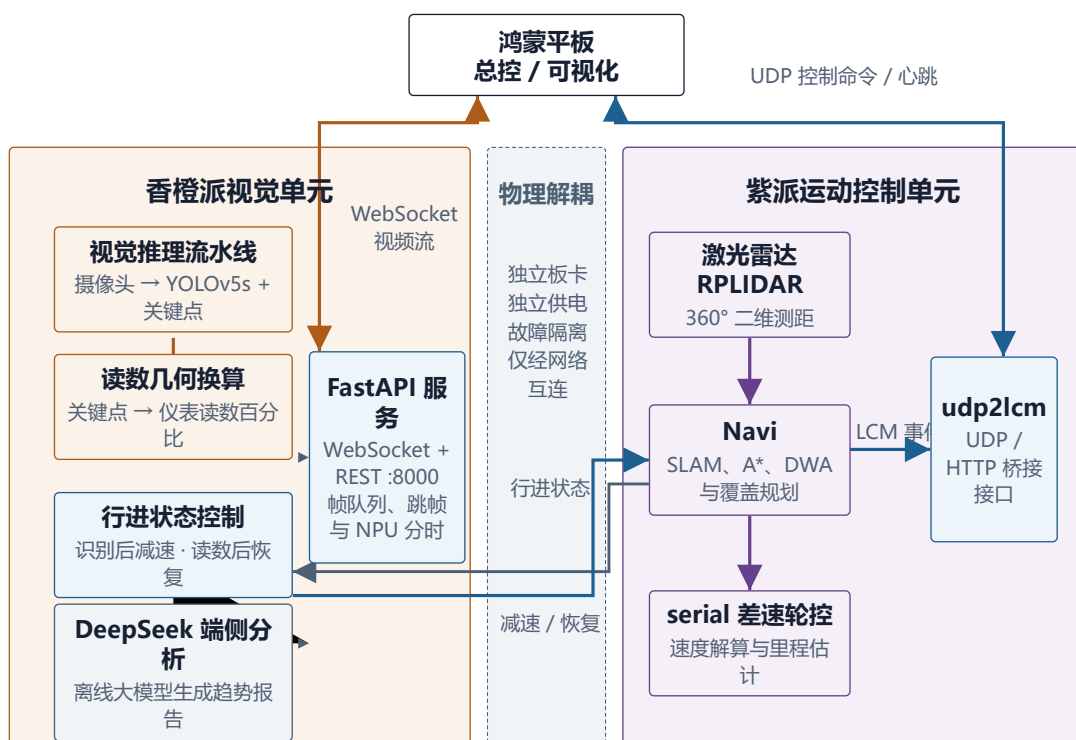


图 10.1 视觉-导航物理理解耦的双单元架构

10.3.2 四层通信方式与 9 字节控制协议

系统通信划分为四层：设备内 LCM 总线（ttl=0 实现进程隔离）、控制 UDP 通道（平板与车之间，9 字节定长包）、数据 HTTP/WebSocket 通道（地图与视频流）、协同通道（软总线 DDO 负责任务协调，车与车多播 LCM 负责执行层避障）。各层在通信范围、协议类型与数据粒度上相互独立，使多设备数据流具有清晰的层次结构。

平板与车之间的控制协议固定 9 字节、大端字节序，命令码对齐 ASCII 编码（如 'f'=102 表示启动全覆盖），便于跨语言实现时保持解析一致。协议内嵌安全机制：3

秒无控制指令即车端急停；发现命令（0x06）仅回响应而不建立控制会话，避免广播发现误触发其他车辆。以极小的包体积表达完整的控制语义，是该协议的主要特点。

10.3.3 ZMAP1 位打包压缩地图

占据栅格地图体积大、拉取慢，是弱网环境下的实际瓶颈。ZMAP1 将每格从 1 字节压缩至 1 bit、每 64 格打包为一个 64 位整数，使地图体积降至约 1/8；获取流程采用“压缩图优先、失败回退普通图”的策略，在保证可用性的同时显著提升了大块地图的传输效率（详见第 8.2 节）。

10.4 自主导航算法链

10.4.1 多分辨率扫描匹配 SLAM

建图不依赖 ROS 建图模块：紫派以单雷达扫描帧与轮控里程估计为输入，采用多分辨率扫描匹配在预测位姿附近搜索最优匹配，并将每帧激光增量融合至概率栅格地图，在 OpenHarmony 上完成了从雷达采集、坐标变换、地图更新到持久化的完整链路。

10.4.2 KLD 自适应粒子滤波定位

定位链路融合轮控里程估计、激光扫描匹配与粒子滤波三类信息源：轮控提供连续运动先验，扫描匹配校正位姿，粒子滤波抑制长期漂移并量化定位不确定性；配合角度归一化与匹配跳变检测，提升了定位异常时的可诊断性。

10.4.3 A* 代价规划与 DWA 局部避障

A* 在经静态障碍膨胀的代价栅格上搜索路径，并融合激光与视觉动态障碍信息，使路径主动远离高风险区域；DWA 在速度空间采样候选 (v, w) ，通过短时轨迹预测评估安全性、目标朝向与速度效率，选出最优速度输出。

10.4.4 多策略全覆盖路径规划

覆盖规划按任务类型组合多种策略：单车支持 zigzag 牛耕式与 STC 生成树覆盖，多机矩形覆盖优先采用 BCD 生成连续路径，失败时回退矩形 zigzag。执行中提取方向变化点作为导航目标，逐段交由 A* 与 DWA 完成，兼顾覆盖完整性、执行可行性与故障降级能力。

10.5 多机协同创新

10.5.1 DDO 共享任务黑板

多机任务协调采用分布式数据对象（DDO）构建共享任务黑板，替代远程拉起式协作：平板写入任务阶段与区域分配，车载 agent 监听与本车相关的状态变化并翻译为本机命令。任务协调层与 C++ 导航进程边界清晰，软总线的状态同步不会直接干

扰实时运动控制。

10.5.2 幂等的车载协同 agent

车载 agent 把黑板上的期望状态转化为本车应执行的命令序列，并通过状态签名比较实现幂等门控：与本车相关的黑板内容未变化时不重复下发命令，避免向导航进程重复发送命令造成干扰。这一设计使多机任务编排简单可靠、易于维护。

10.5.3 空间分区与 COOP_AVOID 协同避障

双车覆盖采用空间分区策略，在任务规划层面减少路线交叉；交界处的偶发冲突由独立的 COOP_AVOID 多播通道兜底，通道内嵌 ACK 确认、重发与超时回退机制，并以 robot_id 仲裁避免对称死锁。协同状态经心跳回传，使平板能区分正常协同停等与异常掉线。

10.6 视觉与端侧大模型的结合

10.6.1 两段式可解释读数路线

仪表读数采用“目标检测 + 关键点定位 + 几何换算”两段式路线：深度学习只负责空间定位（检测框与 4 个关键点），读数换算由确定性的角度比例公式完成。读数过程可解释、可复现，适配不同量程仪表只需更换标定参数。把可学习模块与可解释模块的边界划在恰当位置，是视觉子系统的主要设计考虑。

10.6.2 视觉与大模型分时共享 NPU

在同一块昇腾 NPU 上，通过“暂停视觉推理 → 执行大模型 → 恢复视觉推理”的分时调度与推理锁互斥，使实时视觉推理与 DeepSeek 大模型分析两个原本争抢算力的任务得以共存，机器人本体由此同时具备实时读数与智能分析能力。

10.6.3 端侧大模型工程化

为使 15 亿参数的大模型在边缘 NPU 上稳定运行，系统综合采用了 MindSpore 图模式 O2 级 JIT 编译、StaticCache 预分配 KV 缓存、Top-p 采样与重复惩罚控制生成质量、SSE 流式输出改善交互体验等手段，使端侧大模型从理论可行推进至工程可用。

综上，本项目的创新覆盖国产化自主、异构架构与通信协议、自主导航算法链、多机协同、视觉与端侧大模型结合五个维度，构成一套全栈国产自主的工业巡检机器人系统，可为同类系统的国产化落地提供工程参考。

第十一章 相关工作与对比

本章将本系统与工业巡检机器人现有方案进行对比，明确技术定位与差异化特征。对比数据来自公开行业资料。

11.1 现有方案概况

工业巡检机器人是已有成熟商业产品的领域，应用集中于电力系统巡检。据公开资料与国网、南网招标结果，代表性厂商包括亿嘉和、申昊科技、朗驰欣创、浙江国自与鲁能智能等，产品具备红外测温、局部放电检测与表计识别等功能。这类方案的共性特征是：以电力固定设施为主要场景；部分仍依赖预设轨道或固定巡检点位；产品形态为面向特定行业的封闭商业系统，其操作系统与算法体系的自主可控程度未对外公开。

11.2 技术维度对比

表 11.1 从十一个维度将本系统与典型电力巡检机器人方案进行对比。

表 11.1 本系统与现有方案的逐项对比

对比维度	现有方案（电力巡检机器人）	本系统
操作系统	Linux/Android 或闭源自研，自主可控程度不透明	OpenHarmony 5.0，国产分布式操作系统
AI 算力	通用 GPU 或境外芯片	华为昇腾 NPU，国产边缘算力
算法栈	多基于 ROS 或闭源技术栈	核心算法自主实现，无 ROS 依赖
移动方式	部分依赖预设轨道或固定点位	无轨自主 SLAM 导航，适用于任意室内场景
覆盖能力	多为固定点位巡检	BCD 全覆盖遍历，适用于任意非凸形状区域
多机协同	单机作业或中心服务器调度	软总线共享黑板，去中心协同
大模型分析	云端分析或不具备	DeepSeek 端侧离线分析，数据不出本体
数据安全	部分依赖云端传输	全程设备本体离线，支持断网运行
读数可解释性	多为端到端黑盒识别	两段式几何换算，可标定、可复现
自主可控	部分环节存在外部依赖	内核、算力、语言、算法、硬件全栈国产化
可扩展性	面向特定行业的封闭产品	模块化架构，支持多模态扩展

11.3 差异化特征

基于上述对比，本系统的差异化特征可归纳为四点。

第一，全栈国产化与技术自主。现有方案的底层技术栈对境外操作系统、芯片与 ROS 生态的依赖程度通常不透明，存在供应链安全风险；本系统从操作系统、AI 算力、开发语言、核心算法到硬件平台形成完整的国产化路径，在信创替代的政策背景下具有体系结构层面的差异。

第二，无轨自主全覆盖。部分现有方案依赖预设轨道或固定点位，部署成本高、灵活性有限；本系统采用无轨 SLAM 导航配合 BCD 全覆盖，无需改造现场即可在任意室内场景自主建图并生成不重不漏的覆盖路径，对含柱体与设备的非凸场地同样适用。

第三，端侧感知与分析一体化。多数现有方案的仪表识别停留在“检测并上报”，后续分析依赖云端或人工；本系统通过 NPU 分时调度，在同一块昇腾芯片上同时承载实时视觉推理与离线大模型分析，数据全程不出本体。

第四，去中心化多机协同。现有多机方案多为单机作业或中心服务器调度；本系统基于鸿蒙软总线的共享黑板与车载 agent，实现了无中心服务器、由状态变化驱动的多机协同。

需要说明的是，现有商业产品经过大规模部署验证，在防护等级、行业认证与长期稳定性方面具有明显优势。本系统作为面向技术验证的原型，其价值在于验证了“全栈国产化 + 无轨全覆盖 + 端侧大模型 + 软总线协同”路线的工程可行性，为自主可控巡检机器人的产业化提供技术参考。

11.4 政策契合度

本系统的技术路线与多项国家政策导向契合（表 11.2）。

表 11.2 技术路线与政策导向的契合点

政策导向	契合点
《“十四五”机器人产业发展规划》重点发展特种机器人	本系统面向工业巡检、危险环境作业场景
2025 年政府工作报告首次提出“具身智能”	系统集成自主移动、环境感知、视觉智能与端侧大模型
信创替代：2027 年央国企关键软硬件国产化	全栈国产化路线可直接服务电力、能源等信创场景
昇腾 CANN 开源与 OpenHarmony 生态建设	基于昇腾边缘算力与 OpenHarmony 完成原生部署

综上，本系统在全栈国产化、无轨全覆盖、端侧感知-分析一体化与去中心化协同四个方面与现有方案形成明确差异，且与国家产业政策方向一致，具备进一步工程化与产业化的价值。

第十二章 总结与展望

12.1 工作总结

本项目面向工业巡检场景，在不依赖 ROS 及任何境外机器人中间件的前提下，基于国产软硬件平台设计并实现了一套涵盖自主导航、视觉感知、多机协同与人机交互的完整机器人系统。主要工作如下。

系统架构方面，提出三节点异构协同方案：每辆车由负责运动控制的紫派（Open-Harmony 5.0）和负责视觉智能的香橙派（昇腾 310B）组成，鸿蒙平板（ArkTS）负责多车总控与人机交互；视觉与导航在硬件层面解耦，并通过车内网络交换减速、恢复指令与行进状态。自主导航方面，在紫派上自主实现了雷达驱动、多分辨率扫描匹配 SLAM、A* 全局规划、DWA 局部避障、BCD/STC/zigzag 全覆盖与差速轮控，全部核心代码可审查、可追溯。视觉感知方面，采用“检测 + 关键点 + 几何换算”两段式路线实现可解释仪表读数，识别到仪表后联动紫派调整车速，端到端延迟约 34 ms；并通过 NPU 分时调度在同一块昇腾芯片上集成了 DeepSeek 端侧离线分析。多机协同方面，基于鸿蒙分布式软总线构建共享任务黑板，车载 agent 将任务分配翻译为本机命令，双车以空间分区降低冲突、以 COOP_AVOID 直连通道兜底避障。工程方面，设计了 9 字节定长控制协议与 ZMAP1 位打包地图压缩（约 8 倍），并以统一的接口约定保障三端并行开发与稳定集成。

12.2 局限性与改进方向

作为面向技术验证的原型，系统仍存在以下局限，相应的改进方向已初步规划。

多机坐标系统一方面，当前双车协同依赖“同一物理起点、同一朝向出发后归零”的方式，对人工摆位有依赖；后续可引入激光重定位，使从车自主重定位至主车地图坐标系，放宽初始位姿约束。

覆盖算法方面，当前以 BCD 牛耕分解为主方案；已储备的改进包括 cell 排序 TSP 优化、弓字方向自适应、Spiral-STC 与 frontier 在线覆盖，以及更均衡的双车空间分区。

建图与定位方面，可引入基于 log-odds 与时间衰减的概率建图，使消失的动态障碍被逐步擦除；并将对方车辆作为动态障碍掩膜，避免双车同场建图时的“幽灵墙”问题。

大模型推理方面，端侧模型首次 JIT 编译约 140 s，可通过编译缓存持久化与 INT8

量化优化首响应延迟。

数据传输方面，ZMAP1 已实现约 8 倍压缩，后续可叠加游程编码、增量瓦片传输与 HTTP gzip 压缩，将地图获取延迟进一步压缩。

12.3 后续研究方向

基于本项目的技术基础，后续可在三个方向展开。

第一，面向特定行业的工程化适配。全栈国产化路线使系统具备部署于电力、能源等信创场景的条件，后续可针对具体行业需求在防护等级、通信可靠性与运维流程上工程化改造。

第二，多模态巡检能力扩展。当前以视觉仪表读数为单一巡检模态，模块化架构为扩展红外测温、声纹局放检测、气体浓度监测等传感模态提供了结构基础。

第三，技术路线的验证与推广。本项目验证了“国产操作系统 + 国产 AI 算力 + 自主核心算法 + 端侧大模型 + 软总线协同”路线在工业巡检场景的工程可行性，这一“自主可控与边缘智能深度融合”的思路对其他具身智能应用同样具有参考价值。

参考文献

1. 高亚男. 清洁机器人全覆盖导航技术研究 [D]. 济南: 山东大学, 2016.
2. 赵桐. 基于 ROS 的智能全路径覆盖机器人系统设计与实现 [D]. 哈尔滨: 哈尔滨工业大学, 2018.
3. 唐文字. 基于 ROS 的全路径覆盖机器人的系统设计与实现 [D]. 武汉: 华中科技大学, 2019. DOI: 10.27157/d.cnki.ghzku.2019.002685.
4. 李金澎. 基于分布式控制的移动机器人实时路径规划问题研究 [D]. 北京: 北京邮电大学, 2024. DOI: 10.26969/d.cnki.gbydu.2024.000864.
5. 杨振, 李俊丽, 杨立炜, 等. 安全性 A* 融合 DWA 的分布式多移动机器人路径规划方法 [J]. 控制工程, 2024, 31(12): 2284–2295. DOI: 10.14107/j.cnki.kzgc.20220516.
6. 李艳, 刘丹, 田小东, 等. HarmonyOS 特点与应用前景分析 [J]. 通信与信息技术, 2019(5): 85–87.
7. 鸿蒙应用程序开发 [J]. 计算机教育, 2024(4): F0003.
8. 李竞择, 范承宇, 欧阳迪. 一种开源鸿蒙下基于 Docker 的 Web 开发流程 [J]. 机电产品开发与创新, 2024, 37(2): 51–53.
9. 黄鼎键, 杨华山, 钟勇, 等. 基于 ROS 和激光雷达 SLAM 的动态避障研究 [J]. 自动化仪表, 2024(5): 30–34.
10. 覃雄, 陈家锐. 激光 SLAM 在 AGV 平台构建地图及处理异常的应用 [J]. 装备制造技术, 2024(3): 105–107, 130.

附录 A 术语表

表 A.1 术语表

术语	全称与含义
OpenHarmony	开源鸿蒙，我国自主研发的分布式操作系统
分布式软总线	鸿蒙系统中实现多设备逻辑互联的底层通信能力
DDO	distributedDataObject，分布式数据对象，支持跨设备自动同步的共享数据实体
昇腾 / Ascend	华为自主研发的 AI 芯片系列；本项目采用 310B 边缘 NPU
ACL	Ascend Computing Language，昇腾运行时编程接口
OM	Offline Model，昇腾离线模型格式（由 ATC 编译器生成）
AIPP	AI Pre-Processing，昇腾硬件预处理单元
ATC	Ascend Tensor Compiler，将模型转换为 OM 格式的编译工具
MindSpore	华为开源的国产深度学习框架
LCM	Lightweight Communications and Marshalling，基于 UDP 组播的轻量级发布/订阅通信中间件
SLAM	Simultaneous Localization And Mapping，同步定位与建图
CSM	Correlative Scan Matching，相关性扫描匹配算法
MCL / AMCL	（自适应）蒙特卡洛定位，即基于粒子滤波的定位方法
KLD	Kullback-Leibler Divergence，用于粒子滤波自适应采样中的粒子数量控制
A* / D*	启发式最短路搜索算法 / 增量式重规划算法
DWA	Dynamic Window Approach，动态窗口法，用于局部动态避障
BCD	Boustrophedon Cellular Decomposition，牛耕式单元分解，用于全覆盖路径规划
STC	Spanning Tree Coverage，基于生成树的覆盖算法
ArkTS / ArkUI	鸿蒙应用开发语言 / 声明式 UI 框架
ROS	Robot Operating System，机器人操作系统（本项目未使用）
ZMAP1	本项目设计的位打包压缩地图格式
COOP_AVOID	车与车直连的协同避障多播 LCM 通信通道
信创	信息技术应用创新，即国产自主可控信息化产业